



南開大學
Nankai University

计算机学院

并行程序设计期末综合实验报告

ANN 近似最近邻搜索

姓名：柯云超

学号：2413575

专业：计算机科学与技术

2026 年 7 月

目录

摘要	2
1 引言	2
2 ANNS 查询流水线与代价模型	4
3 四架构并行化：从指令级到设备级	6
4 扩展实验：预算自适应、OPQ 与异构协同	14
5 结果对比与瓶颈分析	20
6 讨论与工程取舍	24
7 结论与实践认识	26
A 数据口径与复核条件	29
AI 使用报告	29

摘要

近似最近邻搜索 (ANNS) 旨在高维向量集合中以较低查询延迟返回足够接近精确最近邻的结果, 通常使用 latency 与 recall@k 同时评价。本文以 DEEP100K[1] ($N = 100000, d = 96$) 为数数据集, 围绕同一查询流水线组织实验: 先抽象出距离计算降本与访问点数削减两类优化, 再分析 AVX2、Pthread/OpenMP、MS-MPI 与 CUDA/cuBLAS[9] 在这两类优化上的适配关系, 最后用 Latency-Recall 曲线、Roofline[8] 与 profile 证据解释性能差异。

实验内容包括 Flat-AVX2、SQ/PQ/FastScan、IVF/IVF-PQ、HNSW、MPI 候选协议、GPU GEMM/设备端 Top- k , 并进一步考察预算自适应 IVF-PQ、OPQ/IVF-OPQ、Roofline/profile 瓶颈分析和 GPU 粗排-CPU AVX2 精排实验。代表性结果显示, Flat-AVX2 将串行 Flat 从 4571.62 us/query 降至 1756.11 us/query, 多核 IVF-PQ 在 Recall@10=0.95945 时达到 57.61 us/query, GPU cublas_tree 在 Recall@10=0.99995 时达到 78.68 us/query。预算自适应 IVF-PQ 在同一实现内将固定 ($nprobe, p$) = (4, 1000) 的 Recall@10 从 0.95945 提升到 0.96520, 并将 latency 从 229.06 us 降至 186.54 us; OPQ 在 $M = 8, p = 1500$ 下将 PQ 的 Recall@100 从 0.95939 提升到 0.97316; 异构实验则表明, 若完整得分矩阵回传主机, CPU 精排收益会被 D2H 与候选选择开销抵消, 跨设备协同必须先和设备端压缩候选。

关键词: ANN; SIMD; Pthread/OpenMP; MPI; CUDA; PQ; OPQ; HNSW; Roofline; 异构协同

1 引言

1.1 问题背景与评价目标

最近邻搜索 (NNS) 的目标是在高维空间中识别与查询向量最接近的向量。图像、文本、音频等非结构化数据通常可以被深度学习模型转换为高维向量, 并在向量距离上保留一定语义相似性, 因此向量检索广泛出现在推荐系统、搜索引擎和大模型相关应用中。随着底库规模和向量维度增加, 精确扫描的计算复杂度过高, 近似最近邻搜索 (ANNS) 转而在保持较高准确率的同时降低查询时间。

给定 d 维底库 $X = \{x_i\}_{i=1}^N \subset \mathbb{R}^d$ 、查询向量 $q \in \mathbb{R}^d$ 和距离函数 $\delta(x, y)$, 精确 kNNS 需要返回集合 S , 满足 $|S| = k, S \subseteq X$, 且任意 $x \in S, y \notin S$ 都有 $\delta(x, q) < \delta(y, q)$ 。ANNS 不要求返回集合完全等于精确 top- k , 而是用召回率衡量近似结果与精确结果的重合程度。DEEP100K[1] 的向量维度为 96, 实验框架采用内积距离

$$\delta(x, q) = 1 - \sum_{j=1}^{96} x_j q_j.$$

精确 Flat 搜索需要扫描全部 N 个向量, 计算量与访存量均随 Nd 线性增长。本文统一使用 Recall@10/Recall@100、单查询 latency、QPS 与相对串行 Flat 的 speedup 作为指标; 比较加速比时, 优先在相近 recall 水平下比较 latency。

ANNS 的优化可以分为两个方面: 一是加快 $\delta(x, y)$ 的计算速度, 二是减少每个 query 需要访问的底库点数。SIMD 主要降低单次距离计算的常数; 多核和 MPI 通过查询、簇或底库分片扩大并行粒度; GPU 将大批量查询转化为矩阵乘以提高吞吐; IVF[2]、HNSW[5] 与 PQ[3] 则从算法层面减少候选数量或压缩候选表示。只有将这些手段放入同一个代价模型, 才能解释为何某些优化在一个架构上有效、在另一个架构上收益有限。

1.2 实验设计

实验分为两组。第一组包括 Flat、SQ、PQ/FastScan[6]、IVF/IVF-PQ、HNSW、MPI 候选协议以及 GPU GEMM/IVF grouped[7] 等基础路径，用来比较指令级、线程级、进程级和设备级并行的不同作用；第二组围绕预算自适应 IVF-PQ、OPQ/IVF-OPQ[4]、GPU 粗排-CPU AVX2 精排和 Roofline/profile 分析展开，用来观察 query 难度、量化误差、候选压缩位置与跨设备数据边界对系统性能的影响。两组实验共同服务于同一问题：在固定数据集和评价指标下，判断不同并行后端分别压低哪一类代价，又把新的瓶颈推向何处。

性能评测以同一台 i9-13900H + NVIDIA GPU 为主要平台。同平台测量点进入横向比较；非同平台或跨 ISA 结果仅作为旁证，不承担主结论。提交口径仍保持原始框架的输出约定，以 average recall 与 average latency 作为可核查指标。

1.3 查询流水线上的实验分组

围绕 ANN 查询流水线，本文把实验分成四类。第一类是**距离计算内核**，包括 Flat-AVX2、SQ、PQ 与 FastScan，用于回答“同一距离函数如何降低单点计算常数”。第二类是**访问点削减与调度**，包括 IVF、IVF-PQ、HNSW 及其分布式变体，用于回答“候选空间减少后，线程、进程和 rank 如何分工”。第三类是**批量矩阵化**，包括 GEMM、cuBLAS、设备端 Top- k 与 grouped-IVF，用于回答“GPU 何时能把 ANN 查询转化为高吞吐矩阵计算”。第四类是**进一步实验**，包括预算自适应 IVF-PQ、OPQ/IVF-OPQ 与 GPU-CPU 异构协同，用于检验 query 难度差异、量化误差和设备边界这几个更接近系统层面的因素。

因此，报告的组织顺序不是 SIMD、多核、MPI、GPU 的简单排列，而是先给出查询流水线和代价模型，再分别讨论距离计算、候选削减、GPU/异构路径，最后用横向评测和 Roofline 解释不同架构的主导代价变化。这样的结构使各类测量点都能落在同一系统模型中，而不是彼此独立的实验片段。

1.4 平台与测量协议

表 1 给出实验环境与计时口径。CPU 侧使用 13th Gen Intel Core i9-13900H，物理核心数 14、逻辑线程数 20，编译选项固定为 `-O3 -mavx2 -mfma -fopenmp`；GPU 侧使用 NVIDIA GeForce RTX 4060 Laptop GPU，显存 8188 MiB，驱动版本 572.83，CUDA 编译器版本为 12.4。横向比较只采用与当前数据集、距离函数和计时定义一致的代表测量点。

数据读取与输出口径沿用实验框架。底库文件为 `DEEP100K.base.100k.fbin`，查询文件为 `DEEP100K.query.fbin`，ground truth 文件为 `DEEP100K.gt.query.100k.top100.bin`；`base` 与 `test_query` 均为连续 float 数组，每 96 个 float 表示一个向量。搜索函数只使用 `base` 与 `query`，ground truth 仅用于评估 recall，不参与候选生成。代表性提交实现保持替换 `flat_search` 调用的接口习惯，返回值格式与原框架一致；辅助文件保持 header-only 形式，输出仍以 average recall 与 average latency 为主。

项目	配置与口径
数据集	DEEP100K, base 数 100000, query 取前 2000 条, 维度 $d = 96$, ground truth 为 top100。
CPU 平台	13th Gen Intel Core i9-13900H, 14 cores / 20 threads, AVX2/FMA。
GPU 平台	NVIDIA GeForce RTX 4060 Laptop GPU, 8188 MiB, driver 572.83, CUDA 12.4。
编译工具	MSYS2 g++ 14.2.0; CUDA 12.4 与 cuBLAS。
框架口径	base/query 为连续 float 数组, 每 96 个 float 表示一个向量; ground truth 只用于 recall 评估。
输出口径	搜索函数替换 <code>flat_search</code> 调用, 返回 top- k id; 输出 average recall 与 average latency。
计时口径	OPQ 表中 latency 采用 warmup 后 3 次重复的 scan+rerank 中位数; train/build/rotation 单独列出, 并不并入在线搜索延迟。GPU 异构实验输出 H2D、GEMM、D2H、候选选择与 CPU rerank 分项。
测量范围	横向比较采用与 DEEP100K、IP 距离和在线 latency 定义一致的代表测量点; OPQ/IVF-OPQ、GPU-CPU 异构与瓶颈分析保留分项结果。

表 1 实验平台与测量口径

进一步实验采用固定参数组和分项计时口径。预算自适应 IVF-PQ 固定 $nlist = 16$, 以 query 到最近两个 coarse centroid 的 margin 划分 easy/medium/hard 三类 query, 并在 $(nprobe, p)$ 上分配不同预算; OPQ 主实验扫描 $M = 8$ 、 $nprobe \in \{2, 4, 8\}$ 与 $p \in \{500, 1000, 1500\}$, 随后补充 $M \in \{12, 16\}$ 的窄扫参, 用于验证旋转量化在不同子空间粒度下的趋势。异构实验扫描 $query_n \in \{1, 128, 512, 2000\}$ 与 $p \in \{100, 300, 500, 1000\}$ 。早期单次测试结果不进入正文结论。

2 ANNS 查询流水线与代价模型

2.1 离线建索引、在线查询与评估

ANN 系统可分为三个阶段。离线阶段完成数据预处理与索引构建, 包括 SQ/PQ 码本训练、IVF 聚类、HNSW 图构建以及 GPU 侧必要的数据库重排。在线阶段对每个 query 进行候选生成、距离估计、候选维护和可选精排。评估阶段将返回结果与 ground truth 比较, 计算 Recall@K 并统计延迟。

离线阶段的成本不进入单查询 latency, 但需要在报告中保留, 因为它决定算法是否适合在线更新场景。Flat 与 SQ 的离线成本很低, 主要是数据布局转换; PQ、OPQ 和 IVF-PQ 需要 KMeans 或旋转矩阵训练, 适合底库相对稳定的批量检索; HNSW 图构建更重, 但在线访问点数较少; GPU 版本还需要在设备侧形成连续 base 矩阵或 IVF list 布局。在线阶段只统计完成一次查询或一个 query batch 的搜索代价, GPU 结果同时保留 CUDA event 的 online time 和包含主机端准备的 wall time。

召回率采用 ground truth 文件计算。返回集合为 R_i , ground truth 的前 K 个 id 为 G_i^K , 则

$$\text{Recall@K} = \frac{1}{Q} \sum_{i=1}^Q \frac{|R_i^K \cap G_i^K|}{K}.$$

本文保留 Recall@10 与 Recall@100 两个口径。Recall@10 与原始输出格式一致, 便于统一校验; Recall@100 更适合观察 PQ/OPQ 粗排加 rerank 的候选覆盖能力。OPQ 与异构实验均在最终 rerank 阶段使用原始 float 向量计算内积距离, 因此召回变化主要反映候选生成质量, 而不是最终距离函数改变。

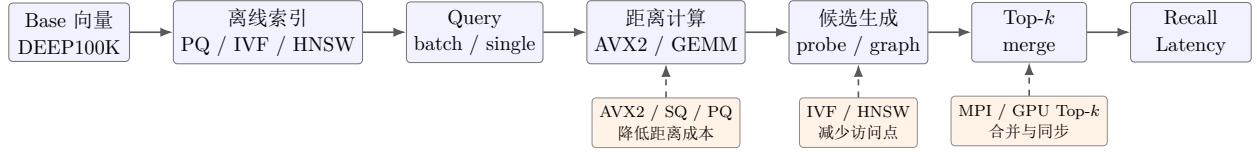


图 1 ANN 查询流水线与不同优化手段的作用位置

2.2 端到端代价模型

为进行瓶颈归因，一次在线查询的端到端延迟可近似分解为

$$T_{\text{online}}(q) \approx T_{\text{score}}(q) + T_{\text{candidate}}(q) + T_{\text{topk}}(q) + T_{\text{sync}}(q) + T_{\text{transfer}}(q).$$

T_{score} 对应向量内积或查表累加； $T_{\text{candidate}}$ 对应 IVF probe、HNSW 图扩展或 PQ 粗排； T_{topk} 对应候选维护与最终合并； T_{sync} 对应线程、rank 或设备端同步； T_{transfer} 对应 MPI 消息或 CPU-GPU 数据搬运。该式不是严格的运行时恒等式：在单机 CPU 实现中 T_{transfer} 可近似为 0，在 GPU stream 或流水线实现中不同阶段可能重叠；本文只用它标记主要代价来源与临界路径上的瓶颈。四类架构的区别并非是否“并行”，而是主要压低哪一项、又放大哪一项。

该模型也解释了为什么代表测量点可以放到同一坐标系中比较。只要数据集、距离函数和 latency 定义保持一致，不同实现下的单点结果可以视作同一代价函数的观测值。另一方面，不能只比较 speedup：如果一个算法通过降低 Recall 换取低延迟，其系统意义必须放在 Latency-Recall 曲线上判断；如果 GPU 方案在线延迟很低但 wall time 很高，则需要拆分 T_{transfer} 和一次性 setup，而不能只报一个平均数。

该模型还给出了报告组织顺序的理由。SIMD、SQ/PQ 和 GPU GEMM 主要作用于 T_{score} ；IVF、HNSW、OPQ 和 rerank 候选预算主要改变 $T_{\text{candidate}}$ ；多核局部堆、MPI root merge 和 GPU 设备端归并主要进入 T_{topk} ；Pthread/OpenMP schedule、rank 等待和 kernel 内同步对应 T_{sync} ；MPI 消息、H2D/D2H 和 score 矩阵回传对应 T_{transfer} 。一项优化若只压低其中一项，但显著放大另一项，端到端效果就可能不理想。GPU-CPU 异构实验正是这种情况：GEMM 使 T_{score} 很低，但完整 score D2H 与主机端候选选择使 T_{transfer} 和 T_{topk} 急剧上升。

在该模型下，Recall 不是额外附属指标，而是候选生成项的质量约束。IVF 减少访问点时可能漏掉真实近邻，PQ 压缩距离时可能改变候选排序，HNSW 的图扩展预算也会影响覆盖率。因此，各个低延迟点都必须同时报告 Recall；仅当 Recall 接近时，比较 latency 或 speedup 才具有工程意义。这也是本报告把主图设置为 Latency-Recall trade-off，而不是只给四架构加速比柱状图的原因。

代价项	主要影响方法	典型主导代价变化
T_{score}	Flat-AVX2、SQ、PQ、FastScan、GPU GEMM	从标量算术迁移到内存带宽、LUT/cache 或矩阵乘吞吐。
$T_{\text{candidate}}$	IVF、HNSW、OPQ、 n_{probe} 、 ef_{Search} 、rerank p	从全库扫描迁移到候选覆盖率和索引访问不均。
T_{topk}	局部 heap、MPI merge、GPU tree merge	得分计算变快后，候选合并成为串行尾部。
T_{sync}	Pthread/OpenMP schedule、rank 等待、GPU block 同步	并行粒度过细时，调度与等待抵消计算收益。
T_{transfer}	MPI 消息、H2D/D2H、score 回传	边界处若传递完整中间结果，系统重新被搬运主导。

表 2 端到端代价模型与主要优化项的对应关系

3 四架构并行化：从指令级到设备级

ANN 查询的并行边界可以从指令级逐层外移到线程级、进程级和设备级。SIMD 改变单个距离内核的执行方式；Pthread/OpenMP 改变单机多核上的任务划分；MPI 改变底库和候选集合的所有权；GPU 改变得分计算的表达形式。若只分别报告加速比，难以看出这些优化之间的关系；若把它们放入同一查询流水线，则可以看到瓶颈如何从算术、访存、调度、通信逐步迁移到候选维护和设备边界。

本章按架构递进梳理关键实验点。重点不在代码清单，而在每种后端回答了什么问题、留下了什么边界条件，以及这些结论如何进入后续 OPQ、异构协同和跨架构建模。

3.1 SIMD/AVX2：距离内核与量化

SIMD 实验首先确认了 ANN 距离计算的基本瓶颈。串行 Flat 搜索逐个扫描底库向量，对每个 base 执行 96 维内积；在 DEEP100K 上，单 query 需要完成约 9.6×10^6 次乘加并读取连续 float32 数据。手写 AVX2/FMA 版本把 8 个 float 作为一个向量 lane 同时处理，并在内循环中使用向量乘加和水平规约。该路径将串行 Flat 的 4571.62 us/query 降至 1756.11 us/query，说明数据级并行能够直接压低 T_{score} 。

然而，Flat-AVX2 的加速并未接近理论向量宽度上限。原因在于该内核的算术强度较低：每个候选只做有限乘加，却必须读取完整 96 维向量。VTune 与汇编检查显示，内循环已经使用 AVX2/FMA 指令，继续扩大展开或调整 FMA 排列的边际收益有限。该结论在期末报告中非常关键，因为它说明“加快距离计算”并不等价于“堆更多指令”；当连续访存成为主导后，数据表示和访问点数量反而比单条 FMA 更重要。

SQ 与 PQ 是 SIMD 路径的第二层推进。SQ 将 float32 压缩为 uint8，使单个向量的访存量显著下降；代表点在 Recall@10 接近精确结果时达到约 427.01 us/query。PQ 则把向量拆成 M 个子空间，每个子空间用码字表示，查询时先构建 LUT，再对 code 做查表累加。其收益来自把 $O(Nd)$ 的 full-vector 计算变为 $O(NM)$ 的 code scan，但召回率依赖候选预算 p 和子空间划分质量。FastScan 进一步将 4-bit PQ code 与小 LUT 组织成寄存器友好的查表路径，结果显示相对普通 PQ 约有 1.41–1.48 倍提升。

SIMD 实验最终沉淀出三条结论。第一，Flat 向量化是必要基线，但不是最终答案，因为它仍然全量扫描完整向量。第二，量化能够同时降低访存和距离估计成本，但必须在 Recall-Latency 曲线上评价，而不能只看延迟。第三，查表内核的性能很大程度由数据布局、code 打包和 cache/LUT 局部性决定。这三条结论直接支撑后文的 OPQ 实验：OPQ 的价值并不是重新写一个更快的 AVX2 点积，而是在普通 PQ 的子空间切分之前降低量化误差，使 PQ/IVF-PQ 内核在同码长下得到更好的候选覆盖。

SIMD 路径	Recall@10	延迟 (us)	系统含义
Flat-serial	1.00000	4571.62	作为所有 speedup 的统一分母，定义精确扫描成本。
Flat-AVX2	1.00000	1756.11	证明距离内核可被向量化，但仍受访存限制。
SQ-AVX2	0.99995	427.01	证明压缩表示可比单纯 FMA 优化带来更大收益。
PQ / FastScan	随 p 变化	–	提供量化查表与 recall-latency trade-off 的基础。

表 3 SIMD 与量化路径的核心测量点

3.1.1 AVX2 Flat、SQ 与量化扫描补充

Flat 搜索的精确性最高，但每个 query 需要访问 100000×96 个 float。AVX2/FMA 将内积规约展开为 8-lane 向量计算，代表性延迟由串行 Flat 的 4571.62 us/query 降至 1756.11 us/query。该结果说明距离内核具有明确的指令级并行空间，但 2.60 倍加速也表明其很快受到内存带宽与 cache

行利用率限制。

SQ 将 float32 压缩为 uint8，单向量访存量显著下降。已有实验中 SQ-AVX2 在 Recall@10=0.99995 时达到约 427.01 us/query，相对串行 Flat 加速 10.71 倍。该收益并非来自更多浮点计算，而是来自数据表示改变后访存压力下降。

从实现角度看，Flat-AVX2 和 SQ-AVX2 的差异主要在数据路径而非控制流。Flat 内核对 96 维 float 做连续加载和 FMA 规约；SQ 内核先用量化参数把数据映射到较窄表示，再在查询时恢复近似内积。由于 DEEP100K 的维度只有 96，单个向量的计算量并不高，访存字节数、对齐和 cache 行利用率对延迟影响更大。该现象也解释了为什么单纯增加 FMA 展开深度不能无限提高 Flat 性能：当内核已经能较好填满向量执行单元后，下一步收益主要来自减少读取的数据量。

这部分实验还澄清了自动向量化和手写 SIMD 的边界。编译器可以识别规则的点积循环，但 ANN 框架中的 top- k 维护、候选窗口和量化重排会使循环结构变复杂；手写 AVX2 的价值不只在在于使用 `vfmmadd`，还在于控制加载、规约、尾部处理和数据布局。对于 Flat 这样的规则内核，自动向量化可能已经接近可用；对于 PQ/FastScan 这样的查表内核，代码布局、打包方式和寄存器内查表更依赖人工设计。

SQ 的结果也说明，精度损失不一定随着压缩而显著出现。DEEP100K 的向量经过标量量化后，在较小 rerank 窗口内仍能保持 Recall@10 接近 1，这意味着底库中高分候选与普通候选之间存在足够间隔，量化误差没有严重扰乱 top-10 排序。相反，PQ 在小候选预算下更容易掉召回，因为子空间量化误差会累积到粗排阶段。后续 OPQ 的目标正是降低这种子空间误差，使同样的 rerank p 覆盖更多真实近邻。

实现路径	主要收益	局限与后续影响
Flat-AVX2	用 FMA 和 256-bit lane 降低点积常数。	全量 float 访存不变，后续需要量化或剪枝。
SQ-AVX2	压缩表示降低内存流量，保持高 Recall@10。	仍需扫描全库，适合作为高召回 CPU 基线。
PQ-ADC	用 LUT 累加替代 full-vector 点积。	召回受子空间量化误差影响，引出 OPQ。
FastScan	用打包 code 和小 LUT 优化查表常数。	更依赖码本设置和高 recall 区间的候选预算。

表 4 距离计算加速路径的收益与局限

3.2 多核 Pthread/OpenMP: IVF/IVF-PQ/HNSW 与调度

多核实验建立在 SIMD 内核之上，但它回答的问题不同。SIMD 只改变单个 worker 如何计算距离，多核实验则决定多个 worker 如何共享 query、候选列表和索引结构。若把每个 query 看成一个独立任务，inter-query 并行最自然：不同线程处理不同 query，线程之间几乎没有共享状态，最后只需汇总各自的输出。该策略在 batch 足够大时稳定，适合前 2000 条 query 的批量评测。

Intra-query 并行则尝试把同一个 query 内的 base、倒排表或候选块拆给多个线程。它可以降低单 query 的响应时间，但会引入局部 Top- k 合并、共享队列、候选去重和同步等待。对 Flat 或 SQ 这类规则扫描而言，intra-query 分块比较直接；对 IVF 和 HNSW 而言，倒排表长度不均、图搜索路径依赖和候选队列维护会迅速放大同步成本。因此，多核路径的核心不是“线程越多越快”，而是识别什么任务粒度足够大、足够独立，能够覆盖调度开销。

Pthread 与 OpenMP 的对照也提供了工程层面的启发。Pthread static 划分开销小，但对 query 难度和 list 长度不均敏感；dynamic 使用原子计数器分发任务，能缓解尾部等待，但每个任务都要付出原子操作成本；thread pool 避免反复创建线程，但队列同步在细粒度任务中可能成为新瓶颈。OpenMP 的 `parallel for` 和 `schedule` 策略简洁，但 runtime 开销较难被单独剥离。实验结果显示，在 IVF-PQ 这样的中等粒度任务上，线程池或 dynamic schedule 更稳；在 PQ 等 query 耗时

较均匀的场景中，静态划分的低常数项可能更有优势。

多核实验还把 HNSW 放入横向比较。HNSW 的图搜索能够显著减少访问点，但其搜索过程具有强数据依赖：候选优先队列和 visited 集合决定下一步扩展位置。多入口点和边划分可以提高 recall，但往往以额外搜索工作换取更完整的候选覆盖，而不是线性降低延迟。这一点在期末报告中被保留下来，用于解释为什么图索引在 trade-off 图中不应只按“快或慢”评价，而应看其在高召回区域的位置。

多核问题	代表现象	系统解释
inter-query vs intra-query	inter-query 在批量评测中最稳定；intra-query 可降单 query latency 但合并开销高。	并行粒度必须与 T_{topk} 和 T_{sync} 一起评价。
static / dynamic / pool	规则任务偏向 static，不均匀 list 或 query 需要 dynamic/pool。	调度策略本质上是在低开销和负载均衡之间取舍。
IVF-PQ 多核路径	代表点达到 57.61 us/query，Recall@10 约 0.95945。	候选削减与多核调度结合后形成低延迟 CPU 方案。
HNSW 并行化	多入口或边划分可提高 recall，但图路径依赖强。	图索引主要优化访问点数量，而非天然适合细粒度并行。

表 5 Pthread/OpenMP 任务调度经验

3.2.1 IVF 与 IVF-PQ

IVF 通过 KMeans 将底库划分为 $nlist$ 个簇，查询时只访问最近的 $nprobe$ 个倒排表。其精度-延迟旋钮清晰：增大 $nprobe$ 会提升召回，但扫描候选数近似线性增加。IVF-PQ 在倒排表内部使用 PQ 粗排与原始向量 rerank，进一步降低候选扫描成本。已有 Pthread/OpenMP 实验中，表 1 平台上的 IVF-PQ 代表点达到 57.61 us/query，Recall@10 约 0.95945，是多核路径中最具性价比的结果之一。

多核实现中，IVF 的主要挑战不是距离内核本身，而是倒排表长度不均。若按 query 静态划分线程，某些线程可能遇到更长 list 或更难 query，导致尾部等待；若按候选块动态划分，调度开销和局部 Top- k 合并次数会上升。Pthread/OpenMP 对比显示，线程池和 dynamic schedule 在中等粒度任务上更稳定，static schedule 在 query 难度差异较大时容易出现尾部空转。图 2 展示 IVF-PQ、HNSW 等索引在 recall 和 latency 之间的不同位置。

若图中区分 global 与 local 码本，global 表示所有倒排簇共享同一套 PQ 码本，local 表示每个簇训练独立 PQ 码本。后者可以降低簇内量化误差，但训练、存储和 cache 压力更高，因此不能只按单点延迟判断优劣。

IVF-PQ 的低延迟来自两层压缩。第一层是 IVF 的 list 裁剪，它把全库访问点从 N 降到若干倒排表之和；第二层是 PQ 的 code scan，它把 list 内候选的 full-vector 距离估计改为 LUT 累加。多核并行再把 query 或 list 级任务分摊到多个 worker。三者叠加后，单点距离内核已不再是唯一瓶颈，候选数量、list 分布和 rerank 窗口共同决定端到端时间。这个现象与 SIMD 实验中的 Flat-AVX2 形成对照：当访问点不变时，内核优化很快撞到访存墙；当访问点被 IVF 减少后，调度和候选维护的重要性上升。

多核实验也说明了参数旋钮的系统含义。增大 $nprobe$ 会提高召回，但会让更多 list 进入扫描；增大 rerank p 会提高最终候选覆盖，但会增加原始 float 精排；增加线程数可以提高吞吐，但在 list 数量小于线程数或任务粒度过细时会带来空转。OPQ/IVF-OPQ 继承了这些旋钮，因此对比时必须保持 M 、 $nprobe$ 和 p 一致。否则，OPQ 的召回提升可能被更大的候选预算混淆，无法说明旋

转本身的作用。

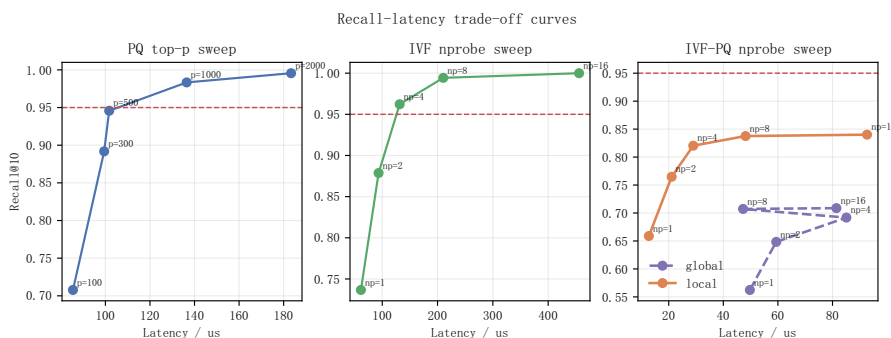


图 2 多核索引路径的 Latency-Recall 曲线

3.2.2 HNSW 变体

HNSW 利用多层小世界图减少访问点,但搜索路径存在强数据依赖,因此并行化难度高于 IVF。实验覆盖了多核 HNSW 三策略,以及 MPI 侧 Block-HNSW、IVF+HNSW 与 HNSW-on-HNSW。它们在结构上体现了同一思想:用图索引减少访问点,再用线程或 rank 扩大候选扩展吞吐。与 IVF 不同, HNSW 的瓶颈通常不是单个距离内核,而是候选队列维护、访问路径依赖与局部负载不均。

HNSW 的并行粒度选择比 IVF 更敏感。若对单个 query 的图扩展做细粒度并行,线程需要频繁竞争候选队列和 visited 标记;若只做 query 间并行,则单 query latency 不会下降。Block-HNSW 与 IVF+HNSW 的思路是在算法层面增加可并行的块或簇,使每个 rank/线程处理相对独立的局部候选,再做全局 merge。其优点是同步边界清晰,缺点是局部图搜索可能错过跨块近邻,需要通过更大的候选预算弥补召回。

HNSW 提供了与 IVF/PQ 不同的索引范式。IVF 是聚类划分,候选集合由 centroid 距离决定;HNSW 是图遍历,候选集合由贪心扩展路径决定。二者都减少访问点,但误差来源不同:IVF 可能因 probe list 不足漏掉边界点, HNSW 可能因搜索路径未扩展到正确邻域而漏掉近邻。多核和 MPI 在这两类索引上面对的负载不均也不同:IVF 的不均来自 list 长度, HNSW 的不均来自路径长度和候选队列状态。

这也是为什么 HNSW 不应被简单地视为“另一个更快的算法”。在某些代表点上, HNSW baseline 的延迟接近多核 IVF-PQ,但 Recall@10 略低;Block-HNSW 或多入口策略可以把 recall 推高,但需要额外搜索和合并。它在期末报告中的价值,是展示访问点削减并不只有聚类倒排表一种方式,同时也提醒并行化需要尊重算法内部的数据依赖。

3.3 MPI: owner-computes 候选协议

MPI 路径把 ANN 问题从共享内存扩展到多 rank 环境。与多线程不同,rank 之间不能共享索引对象和候选队列;每个 rank 只能稳定访问自己的底库分片。因此, MPI 设计首先要回答正确性问题:多个局部 top-k 如何合并成全局 top-k,以及返回结果如何保持全局 id 与 ground truth 对齐。实验采用 owner-computes 策略,每个 rank 独立构建本地 IVF-PQ 或 HNSW 变体索引,在线阶段只返回固定长度的本地候选, root 端执行全局 merge。

该协议的优点是通信量与候选数相关,而不是与底库大小相关。每个 rank 对每个 query 返回 k 个 (distance, id) 候选,消息规模近似 $O(QPk)$,不会把原始向量或大规模 score 矩阵传回 root。已有单机多 rank 结果显示,通信与 merge 占比约为 1% 量级,说明在 DEEP100K 和定长候选协议下, MPI 层没有吞掉本地搜索的收益。该结果也提示后续优化方向:继续压缩通信格式的价值有限,更应关注 rank-local 搜索时间最大值、底库分片是否均衡,以及不同 rank 上图搜索路径是否存在尾部。

MPI 实验中的 Block-HNSW、IVF+HNSW 与 HNSW-on-HNSW 进一步展示了“候选生成

器”和“分布式外壳”的解耦。MPI 层不需要理解局部索引内部细节，只要求每个索引返回可合并的候选数组。这样一来，IVF-PQ 可以作为低延迟局部扫描器，Block-HNSW 可以作为高召回图索引，IVF+HNSW 可以先用簇裁剪再做局部图扩展。不同局部生成器对应不同 Recall-Latency 位置，但它们共享同一套候选协议。

MPI 路径的价值不只是一组加速比，而是提出了一个跨架构通用原则：并行边界处不应传递过大的中间状态，而应传递足够恢复最终结果的压缩候选。这个原则也适用于 GPU-CPU 异构实验。异构路径的慢点并不在 GPU 得分，而在完整 score 矩阵回传 CPU 后重新做候选选择，违反了“边界处只传候选”的原则。MPI 候选协议与异构实验的慢点归因形成了直接对照。

MPI 代表点	Recall@10	延迟 (us)	解释
IVF-PQ 4 ranks $\times$ 2 threads	0.96560	232.45	低通信候选协议下的分片扫描代表点。
Block-HNSW 8 ranks $\times$ 2 threads	1.00000	195.11	用多 rank 图索引合并换取高召回。
comm+merge 占比	—	~1%	当前规模下主要瓶颈仍是 rank-local 搜索。

表 6 MPI 分布式候选协议结果

3.3.1 MPI 候选协议

MPI 路径采用 owner-computes 的底库分片策略，各 rank 只搜索 rank-local 分片并返回定长候选，root 端做全局 merge。已有 MS-MPI 多 rank 结果显示，IVF-PQ 4 ranks $\times$ 2 threads 的代表点为 232.45 us/query, Recall@10=0.96560, 相对串行 Flat 可达到 19.67 倍加速；MPI Block-HNSW 8 ranks $\times$ 2 threads 达到 195.11 us/query, Recall@10=1.00000。两者的 comm+merge 占比约 1%，说明在 DEEP100K 规模下，通信不是主瓶颈；rank-local 搜索和负载均衡仍决定端到端延迟。单节点 MS-MPI 点并不以击败共享内存 OpenMP 为目标，其价值在于验证底库分片、定长候选协议和全局 merge 的正确性；若扩展到跨机器或更大底库，该协议才体现容量扩展意义。

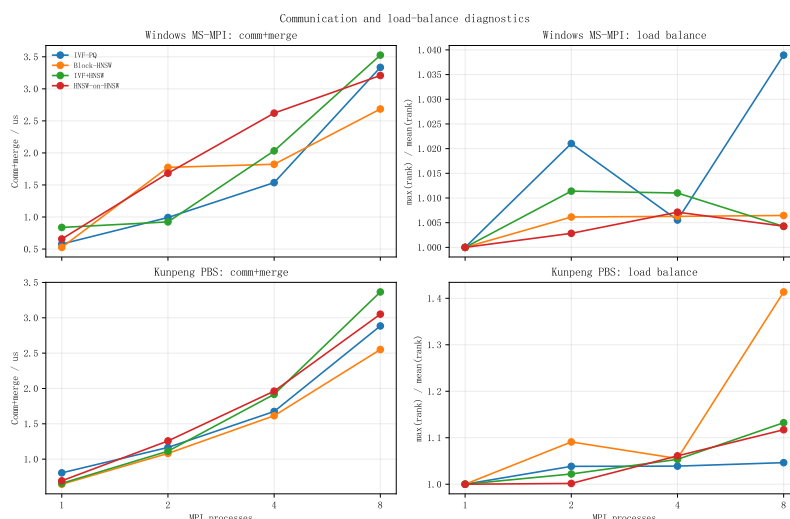


图 3 MS-MPI 候选协议中的搜索、通信与合并占比（跨平台诊断只作为机制旁证）

通信占比低并不意味着 MPI 没有代价，而是说明当前数据规模和候选协议把通信量控制在较小范围。每个 rank 只返回定长候选，消息规模约为 $O(k \cdot r)$ 而非 $O(N)$ ；root 端 merge 的复杂度也与候选数相关。因此，继续压缩通信格式只能带来有限收益，更应关注各 rank 的候选扫描时间是否均衡。若扩展到跨机器集群，网络延迟和数据分片放置会重新变得重要；本报告主结论限定在单机多 rank 实验。

MPI 实验还提供了一个重要的正确性约束。每个 rank 返回的候选必须携带全局 id, root 端合并后才能与 ground truth 对齐；如果局部 id 没有转换, Recall 会被系统性低估或高估。这个细节在共享内存实验中不明显, 因为所有线程天然访问同一个全局数组；到了 MPI 或异构设备边界, id 映射就成为算法正确性的一部分。期末报告保留这一点, 是因为 GPU-CPU 异构精排同样需要在 GPU 粗排候选、CPU 原始向量和最终 ground truth 之间保持 id 一致。

访问点削减方法	主要误差来源	并行化难点
IVF	probe list 不足导致跨簇近邻遗漏。	倒排表长度不均, $nprobe$ 小时 list 级任务数不足。
IVF-PQ	IVF 漏候选叠加 PQ 粗排误差。	code scan、rerank 和 list 调度需要同时平衡。
HNSW	图搜索路径未覆盖真实邻域。	候选队列和 visited 集合依赖强, 细粒度共享困难。
MPI 分片索引	局部候选必须足以恢复全局 top- k 。	全局 id、定长候选协议和最慢 rank 尾部。

表 7 减少访问点类算法的误差来源与并行化难点

3.4 GPU/CUDA: 批量矩阵化与设备端 Top- k

GPU 路径回答了另一个问题: ANN 的全库扫描是否一定要被视为一批独立点积。对于单个 query, GPU kernel 启动、H2D/D2H 和 Top- k 后处理的固定开销很高, 未必比 CPU 有优势；但当 query 形成 batch 后, base 矩阵与 query 矩阵的内积可以改写为矩阵乘法 $S = BQ^T$ 。这使得 query 间并行和 query 内并行同时暴露给 GPU, 计算被映射到高吞吐 GEMM 路径。

GPU 实验逐步比较了手写 tiled GEMM、cuBLAS、线程 0 Top- k 合并和共享内存树形 Top- k 合并。结果显示, cuBLAS 能显著压低 score 计算时间, 但如果 Top- k 仍由单线程顺序合并, 整体收益会被串行尾部吃掉；树形合并降低了设备端后处理开销, 使 cublas_tree 成为最具代表性的 GPU Flat 路径。该路径在高召回下达到 78.68 us/query, 说明 GPU 对规则、密集、批量的得分计算非常适合。

GPU IVF grouped 则尝试把候选削减与矩阵化结合起来。普通 IVF 的每个 query 访问不同 probe list, 导致 batch 内候选集合不规则；grouped-IVF 先按 probe 簇把 query 分组, 使访问同一 list 的 query 形成小矩阵乘。该方法说明 GPU 上的索引加速并非不能做, 而是需要先把不规则访问重新整理成规则批处理。它也暴露了 GPU 路径的核心矛盾: 减少访问点会削弱矩阵乘规模和规则性；保持全库 GEMM 则计算规则但访问点最多。

GPU 路径给出两条重要经验。第一, GPU 的优势依赖 batch 和规则矩阵化, batch=1 的低延迟场景并不天然适合 GPU。第二, GEMM 不是完整 ANN 系统的终点, Top- k 、score 矩阵存储、结果回传和索引分桶都会在 score 阶段加速后成为显性成本。异构实验正是在这两条经验上继续推进: 既然 GPU score 很快, 是否可以让 CPU 做更灵活的精排；实验结果则证明, 如果候选压缩没有在设备端完成, 完整 score 回传会直接抹掉 GEMM 的收益。

GPU 路径	Recall@10	延迟 (us)	系统含义
cublas_tree	0.99995	78.68	高召回批量吞吐代表点, 说明矩阵化得分有效。
IVF grouped $nprobe = 4$	0.96280	145.32	体现候选裁剪与矩阵化规则性之间的折中。
IVF grouped $nprobe = 8$	0.99345	189.90	提高召回需要访问更多 list, 延迟随之上升。

表 8 GPU 批量检索路径的代表性结论

3.4.1 GPU GEMM 与设备端 Top- k

GPU 路径将 query batch 与 base 矩阵做乘法，得分矩阵由手写 tiled GEMM 或 cuBLAS 产生。统一口径采用 GPU-only cublas_tree 在线基线：在 $query_n = 2000$ 时达到 78.68 us/query, Recall@10=0.99995，相对串行 Flat 加速 58.10 倍。该结果说明 GPU 对规则、批量、矩阵化的得分计算非常适合；同时 CUDA event 分段也显示，当 GEMM 被加速后，设备端 Top- k 与结果拷贝会变成显性开销。

GPU 侧比较了手写 16×16 tiled GEMM、cuBLAS、线程 0 合并 Top- k 与共享内存树形合并 Top- k 。在 $query_n = 2000$ 时，手写 GEMM 的 score 阶段约 240.08 ms，cuBLAS 降至约 69.47 ms；线程 0 合并 Top- k 约 226.56 ms，树形合并降至约 91.92 ms。两项优化叠加后形成 cublas_tree 代表点。该对照说明 GPU 优化不能只盯矩阵乘法：得分矩阵越快产生，Top- k 合并、结果回传和 query chunk 管理越容易成为主导项。

GPU IVF grouped 路径则尝试减少访问点。per-query IVF 让每个 query block 扫描自己的 probe list，控制简单但 batch 内候选集合不一致；grouped-IVF 先按 probe 簇重排 query，把访问同一 list 的 query 组成小 batch，再用局部 GEMM 扫描候选。该方法把不规则 probe 访问部分重新转化为矩阵化计算，但需要额外的分桶和重排。已有结果中 grouped-IVF 在 $nlist = 16, nprobe = 4$ 时 Recall@10 约 0.9628，在线延迟约 145.32 us/query； $nprobe = 8$ 时 Recall@10 约 0.99345，延迟约 189.90 us/query，体现了候选裁剪和矩阵化吞吐之间的折中。

GPU 实验与 CPU 多核实验的差异在于，GPU 更难容忍不规则控制流。CPU 上每个线程处理不同长度 list，只要调度策略合理，负载不均仍可被动态分配部分吸收；GPU 上同一个 warp 或 block 内若候选长度差异过大，就会造成分支发散、访存不合并或线程空转。因此，GPU IVF 的核心不是简单把 list 扫描搬到 device，而是先把访问模式整理成适合 GEMM 或至少适合 coalesced load 的形态。grouped-IVF 的意义就在于用分桶换取规则性。

这一点也解释了为什么 GPU Flat 虽然访问点最多，却可能在高召回批量场景中很强。全库 GEMM 不做候选裁剪，但矩阵形态高度规则，cuBLAS 可以充分利用 Tensor Core 或 CUDA core 的吞吐；IVF 减少了候选数量，却破坏了规则性，需要额外分组才能恢复吞吐。换言之，GPU 上“少算”不一定自动更快，只有当少算带来的访问点减少超过分桶、同步和小矩阵低利用率成本时，IVF 才能超过全量 GEMM。

3.5 相通之处与差异

3.5.1 两条优化主线

第一条主线是加快距离计算。Flat-AVX2 直接对 96 维内积做向量化；SQ 将 float32 压缩为 uint8，减少访存量；PQ 将 d 维向量拆成 M 个子空间，距离计算变成 M 次查表累加；FastScan 将部分 LUT 压入寄存器友好的布局；GPU 则把 query 和 base 之间的得分计算改写为矩阵乘。

第二条主线是减少访问点数量。IVF 通过聚类只访问最近的若干倒排表，HNSW 通过图搜索减少全局扫描，IVF-PQ 与 HNSW 变体进一步组合了候选剪枝与压缩表示。并行化的难点随算法改变：IVF 的主要问题是倒排表长度不均，HNSW 的主要问题是搜索路径数据依赖，GPU IVF 的主要问题是 batch 内 probe 集合不一致导致规则矩阵化困难。

两条主线并非互斥。典型高性能 ANN 路径通常先减少访问点，再降低单点距离代价，最后用并行硬件摊薄候选扫描。例如 IVF-PQ 先用 coarse centroid 裁剪 list，再用 PQ LUT 给 list 内候选粗排，最后对有限的 top- p 候选用原始 float 向量 rerank。GPU Flat 则走另一条路径：不减少访问点，而是用大 batch GEMM 把全库扫描推到高吞吐矩阵乘单元，随后用设备端 Top- k 避免完整 score 矩阵回传。二者都可达到较高吞吐，但瓶颈位置完全不同。

多核和 MPI 的作用更偏向并行调度层。inter-query 并行把不同 query 分配给不同线程，结构简单但依赖 batch 大小；intra-query 并行把同一 query 的候选或倒排表分给多个线程，可以降低单

query latency, 但需要合并局部 Top- k 。MPI owner-computes 则把底库按 rank 分片, 每个 rank 返回定长候选, root 端全局归并。上述三种方案的共同约束是尾部负载: query 难度、倒排表长度和图搜索路径都会造成某些线程或 rank 更晚结束。

从算法设计角度看, ANN 并行化不是单一层面的优化, 而是“候选规模”和“候选处理方式”的联合设计。候选规模越大, SIMD/GPU 的规则扫描空间越充分, 但 Top- k 和数据搬运压力越高; 候选规模越小, 在线计算量下降, 但召回更依赖索引结构和粗排质量。PQ、IVF、HNSW、OPQ、GPU grouped-IVF 都是在这张候选规模图上选择不同位置。OPQ 选择不改变在线候选预算的前提下提高候选质量; 异构实验则选择把 GPU 粗排产生的高质量候选交给 CPU 精排, 但结果证明候选压缩位置必须足够靠近 GPU 端。

四种架构的差异还体现在“规则性”的要求上。AVX2 可以处理较规则的连续扫描, 也能承受一定程度的 LUT 查表; Pthread/OpenMP 可以处理更不规则的 query/list 粒度, 但需要避免过细任务; MPI 要求跨 rank 边界的信息足够小且格式固定; GPU 对规则矩阵化最敏感, 不规则 list 或图遍历需要先分桶、重排或转化为 batch。由此可见, 同一种算法在不同架构上并不是简单移植, 而是要重新选择并行粒度和数据布局。

3.5.2 并行粒度映射

关注点	AVX2 SIMD	Pthread/OpenMP	MPI	CUDA GPU
并行粒度	SIMD lane 与展开循环	query、簇、候选分块	rank-local 底库分片	thread block / warp / batch
主要收益	降低距离内核常数	利用多核吞吐	扩展候选扫描容量	大批量矩阵化得分
主要代价	对齐、尾部、查表布局	调度、false sharing、尾部任务	gather/merge、负载均衡	H2D/D2H、Top- k 、occupancy
适配算法	Flat、SQ、PQ、FastScan	IVF、IVF-PQ、HNSW	IVF-PQ、Block-HNSW	GEMM Flat、grouped IVF
主导代价变化	DRAM 到 LUT/cache	计算到调度与不均衡	通信到 rank-local 搜索	GEMM 到 Top- k /transfer

表 9 四类架构在 ANN 查询流水线中的并行粒度与瓶颈

表 9 也可以反向理解为一张设计检查表。若某个方案声称在 GPU 上加速 HNSW, 需要说明图遍历的不规则性如何被处理; 若某个方案声称 MPI 可扩展, 需要说明候选协议是否避免传递大规模中间结果; 若某个方案声称多线程加速 IVF, 需要说明倒排表长度不均如何被调度吸收; 若某个方案声称 SIMD 优化 PQ, 需要说明 LUT 和 code stream 是否适合向量加载或寄存器查表。本文后续各节均按这个标准解释结果, 而不是只报告“某方法更快”。

3.5.3 算法与后端对应关系

表 10 按算法族给出各后端的实现位置。Flat 适合作为距离内核基线; IVF/IVF-PQ 适合观察候选裁剪与调度; PQ/FastScan 体现量化扫描; HNSW 体现图索引访问点削减; OPQ/IVF-OPQ 位于量化前处理位置。不同算法并不都天然适合所有架构, 因此表中只保留能够说明机制的代表实现, 不强行把每个空格都补成一个低质量实现。

算法族	SIMD/AVX2	Pthread/OpenMP	MPI	CUDA GPU
Flat	Flat-AVX2	query 并行基线	分片扫描基线	GEMM + 设备端 Top- k
SQ/PQ/ FastScan	量化扫描核心	query/list 并行	局部候选生成器	候选压缩方向
IVF/IVF-PQ	粗排与精排内核	动态调度与线程池	owner-computes 分片	grouped-IVF GEMM
HNSW 变体	距离内核	多入口/边划分	Block-HNSW 等局部图索引	图遍历不规则, 未作为 GPU 主路径
OPQ/IVF-OPQ	旋转量化	CPU 检索路径	–	异构候选前处理方向

表 10 算法族与后端实现位置

4 扩展实验：预算自适应、OPQ 与异构协同

4.1 预算自适应 IVF-PQ

固定 $nprobe$ 和 rerank 预算 p 的 IVF-PQ 对所有 query 使用同一工作量, 但 DEEP100K 中 query 难度并不均匀。若某个 query 到最近 coarse centroid 与次近 coarse centroid 的得分差距很大, 真实近邻通常集中在少量倒排表内; 若二者差距很小, query 位于多个簇边界附近, 较小 $nprobe$ 更容易漏掉真实近邻。预算自适应 IVF-PQ 利用这一现象, 在不改变 IVF-PQ 索引结构和最终 rerank 距离函数的前提下, 为不同 query 分配不同的 $(nprobe, p)$ 。

具体做法是先用少量校准 query 统计 margin 分位数, 得到阈值 τ_{low} 与 τ_{high} 。在线查询时, 设 s_1, s_2 为 query 与最近、次近 coarse centroid 的内积分数, margin 定义为 $s_1 - s_2$ 。当 margin 小于 τ_{low} 时, 将 query 视为 hard query, 使用较高 probe 预算; 当 margin 大于 τ_{high} 时, 将 query 视为 easy query, 使用较低预算; 其余 query 使用中等预算。分组执行版本进一步把同类 query 合并处理, 减少频繁切换预算造成的调度开销。该策略只改变候选预算, 不改变 PQ code、IVF list 或最终 float rerank, 因此可以直接与固定预算 IVF-PQ 比较。

配置	Recall@10	延迟 (us)	平均 $nprobe$	平均 p	说明
固定 2/500	0.86940	148.43	2.00	500.00	低预算固定点。
固定 4/1000	0.95945	229.06	4.00	1000.00	中等预算固定点。
固定 8/1500	0.99265	340.32	8.00	1500.00	高预算固定点。
保守自适应	0.96520	217.38	3.91	919.20	easy/medium/hard 分配预算。
激进自适应	0.97115	216.00	4.44	953.75	hard query 使用更高预算。
保守自适应 + 分组	0.96520	186.54	3.91	919.20	分组执行后调度开销更低。

表 11 预算自适应 IVF-PQ 与固定预算 IVF-PQ 的对照 ($nlist = 16$, query 数为 2000)

表 11 中最关键的对照是固定 4/1000 与保守自适应 + 分组。二者使用同一索引、同一距离估计和同一最终 rerank 口径; 后者把 623 条 hard query 分配到较高预算, 把 808 条 easy query 分配到较低预算, 使平均 $nprobe$ 从 4.00 降至 3.91, 平均 p 从 1000 降至 919.20。结果是 Recall@10 从 0.95945 提升到 0.96520, 同时 latency 从 229.06 us 降至 186.54 us。这个点没有取代多核 IVF-PQ 的最低延迟代表点; 它说明在同一 IVF-PQ 扫描空间内, 按 query 难度重新分配候选预算可以形成更优的 Recall-Latency 操作点。

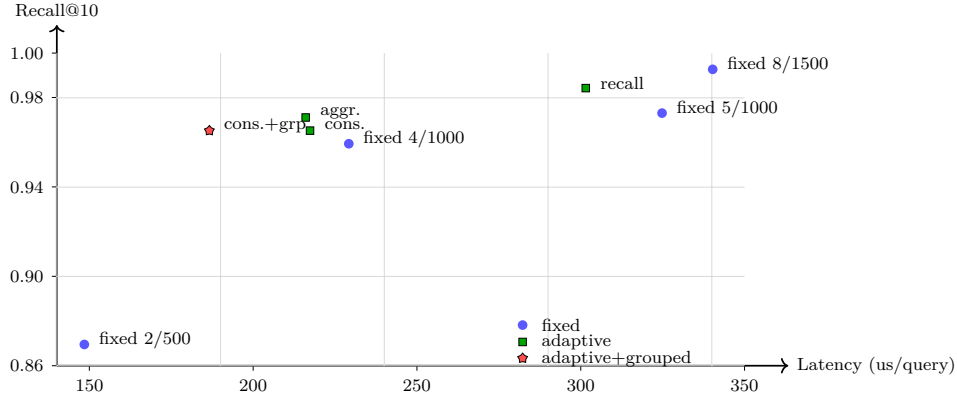


图 4 预算自适应 IVF-PQ 的 Recall@10-Latency 对比

图 4 还显示，自适应策略并不是简单增加平均预算。Recall-oriented 配置可以继续提高召回，但 latency 接近高预算固定点；aggressive 配置在 recall 与 latency 之间更均衡；conservative grouped 则更适合低延迟场景。由此可见，IVF-PQ 的有效优化不只来自更快的扫描内核，也来自候选预算在 query 集合内部的分配方式。该结果为后文的横向汇总提供了一个新的 Pareto 点：它仍位于 CPU/IVF-PQ 路径，但已经不只是固定参数扫描。

4.2 OPQ/IVF-OPQ 旋转量化

PQ[3] 将 96 维向量划分为 M 个子空间并分别量化。查询时先构造 $M \times 256$ 的 LUT，再对每个 base code 做 M 次查表累加，复杂度从 $O(Nd)$ 变为 $O(NM)$ 。FastScan[6] 进一步优化 code 与 LUT 的布局，将查表路径压入 SIMD 更友好的寄存器/packed-code 形式，已有实验显示其相对普通 PQ 约有 1.41-1.48 倍提升。

OPQ/IVF-OPQ[4] 属于量化前处理方向。其动机是普通 PQ 固定按原始维度切分子空间，如果高方差或强相关维度集中在同一子空间，量化误差会变大。实验先学习一个正交旋转矩阵，将 base 与 query 变换到旋转空间，再执行 PQ/IVF-PQ 训练与搜索；最终 rerank 仍使用原始 float 向量，以保证召回指标与原始任务一致。当前实现采用 PCA 正交旋转与子空间方差均衡排列，属于无外部依赖的 OPQ-style 工程近似；完整交替式 Procrustes OPQ 可作为后续替换，但本文评测只讨论该工程近似在当前框架中的效果。

实现包含四个关键步骤。第一，基于 base 向量计算 96×96 协方差矩阵，并用 Jacobi 对称特征分解得到正交基。第二，将特征值从大到小排序，并采用贪心方式把高方差主成分分散到 M 个子空间中，避免某个 PQ 子空间承担过高方差。第三，对 base 和 query 同时应用该正交旋转，保证内积排序在旋转空间中仍与原空间具有一致的几何解释。第四，在 PQ 或 IVF-PQ 粗排得到 top- p 候选后，用原始 float 向量执行 rerank。该设计把变量集中在旋转与训练阶段，距离估计仍采用同一套 AVX2/PQ 搜索路径，便于进行同码长对比。算法流程如算法 1 所示。

算法 1 OPQ-style rotation + PQ/IVF-PQ search

```

1 Input: base X, query Q, subspaces M, candidate budget p
2 C = covariance(X)
3 U, lambda = JacobiEigen(C)
4 perm = GreedyBalance(lambda, M)
5 R = PermuteColumns(U, perm)
6 Xr = X * R; Qr = Q * R
7 Train PQ or IVF-PQ on Xr
8 for each query q:
9     coarse = ADC_or_IVF_ADC(q * R, p)
10    result = RerankWithRawFloatVectors(q, coarse)

```

OPQ 实验在表 1 的 CPU 平台上执行，固定 $M = 8, k_{sub} = 256$ ，查询数为 2000。每个配置先执行 128 条 query warmup，再重复 3 次，表中 latency 为 scan+rerank 的中位数；PQ/IVF 训练、

base/query 旋转和索引构建均作为离线成本单独记录，不并入在线 latency。3 次重复足以排除把训练、旋转或 build 误并入在线查询的口径错误，但不足以支持 OPQ 与 PQ 之间百微秒级差异的速度排序。因此，表 12 中的延迟只用于确认二者处于同一量级；OPQ 的主要收益不是把 PQ 变成更快的扫描器，而是在相同码长和候选预算下提高粗排覆盖率：当 $p = 1500$ 时，PQ 的 Recall@100 为 0.95939，OPQ 提升到 0.97316；IVF-PQ 在 $nprobe = 8, p = 1500$ 时为 0.95228，IVF-OPQ 提升到 0.96568。旋转矩阵只有 96×96 个 float，约 36 KiB，相对 PQ code 和 IVF 索引几乎不改变在线内存阶数。

该 OPQ 系列结果还需要与表 17 中的多核代表点区分口径。表 12 的 IVF-PQ/IVF-OPQ 是单线程、大候选预算 $p = 1500$ 、以 Recall@100 观察候选覆盖的实验；表 17 中 57.61 us/query 的 IVF-PQ 是 Pthread/OpenMP 多核代表点，候选预算和统计口径不同，不能直接解释为同一实现相差 27 倍。

方法	码长 (B/vec)	旋转矩阵	$nprobe$	p	Recall@10	Recall@100	延迟 (us, 约)
PQ	8	—	—	1000	0.98070	0.92202	1854
OPQ	8	36 KiB	—	1000	0.98975	0.94653	1700
PQ	8	—	—	1500	0.99180	0.95939	2298
OPQ	8	36 KiB	—	1500	0.99545	0.97316	2179
IVF-PQ	8	—	8	1500	0.98660	0.95228	1590
IVF-OPQ	8	36 KiB	8	1500	0.99005	0.96568	1078

表 12 OPQ/IVF-OPQ 在同码长下的召回增益 ($M = 8$, latency 为在线 scan+rerank 中位数；小幅延迟差异不作速度排序)

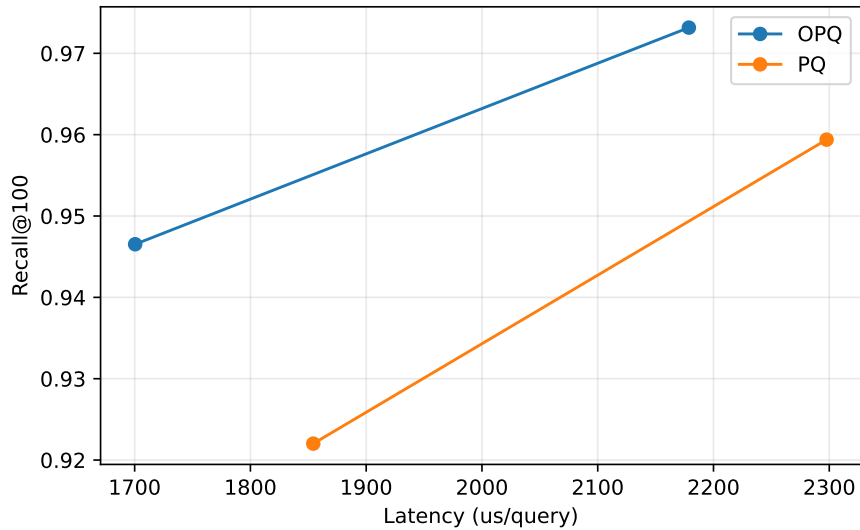
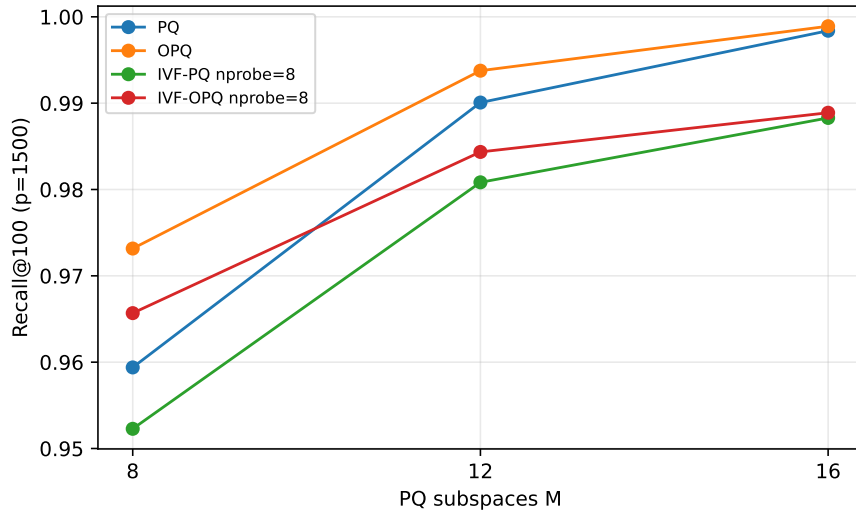


图 5 OPQ 与 PQ 在同压缩率下的 Recall@100-Latency 对比 (在线 scan+rerank 口径)

补充扫描进一步固定 $p = 1500$ ，比较 $M = 8, 12, 16$ 。结果见表 13 与图 6。当 M 从 8 增至 16 时，PQ 的压缩码长增加、每个子空间维度下降，Recall@100 从 0.95939 提升到 0.99839；OPQ 从 0.97316 提升到 0.99891。IVF 路径也呈现类似趋势， $nprobe = 8, p = 1500$ 下 IVF-PQ 从 0.95228 提升到 0.98829，IVF-OPQ 从 0.96568 提升到 0.98889。高 M 区间中 OPQ 的边际收益变小，因为普通 PQ 已经有足够细的子空间划分；低 M 区间中旋转更有价值，因为每个子空间承载更多维度，方差分布不均会直接放大量化误差。表中 latency 对 M 不严格单调，主要因为 $p = 1500$ 下最终 float rerank 占比较高，code scan 的 M 差异被精排和运行抖动部分掩盖。

方法	M	码长 (B/vec)	$nprobe$	Recall@100	延迟 (us, 约)
PQ	8	8	—	0.95939	2298
OPQ	8	8	—	0.97316	2179
PQ	12	12	—	0.99007	1839
OPQ	12	12	—	0.99376	1872
PQ	16	16	—	0.99839	2139
OPQ	16	16	—	0.99891	2141
IVF-PQ	16	16	8	0.98829	1444
IVF-OPQ	16	16	8	0.98889	1526

表 13 不同 PQ 子空间数下的 OPQ/PQ 补充扫描 ($p = 1500$, latency 为在线 scan+rerank 中位数)图 6 M 从 8 到 16 时 OPQ 与 PQ 的 Recall@100 变化

该扫描同时暴露了工程代价。 M 增大后, LUT 累加次数增加, 码本和 code 体积也上升; OPQ 还需要额外的 96 维稠密 query 旋转。在线 latency 已排除训练、base 旋转和索引构建, 因此不会把离线成本误并入单查询延迟。报告中仍不把 OPQ 描述为无条件加速技术, 而将其定位为在固定压缩率或低 M 配置下改善候选覆盖的量化前处理。

OPQ 的结果需要与 PQ/FastScan 放在一起理解。FastScan 优化的是“给定 code 和 LUT 后如何更快扫描”, OPQ 优化的是“给定码长和候选预算时 code 是否更能保留近邻结构”。二者不互相替代, 而是位于 PQ 流水线的不同位置。若工程时间允许, OPQ 旋转后的 code 仍可继续采用 FastScan 风格布局; 本文没有把这两者叠加, 是为了避免实现分支过度扩散。当前结论已经足够说明: 在 $M = 8$ 这种压缩较强的场景中, 旋转前处理能明显提高 Recall@100; 当 $M = 16$ 时普通 PQ 已接近饱和, OPQ 的边际收益自然下降。

方法	训练/构建 (s, 约)	旋转 (s, 约)	索引内存 (MiB)	在线阶段是否计入
PQ $M = 8$	4.3	0.5	0.95	否
OPQ $M = 8$	4.2	0.9	0.95 + 0.04	否
IVF-PQ $M = 8$	4.6	0.5	37.96	否
IVF-OPQ $M = 8$	5.1	0.9	37.96 + 0.04	否

表 14 OPQ 代表配置的离线成本与内存开销 (离线时间为代表性量级, 不用于百毫秒级排序)

4.3 CPU-GPU 异构协同

异构实验使用 GPU/cuBLAS 计算 query-base 得分矩阵，将得分拷回 host 后选取 top- p 候选，再由 CPU AVX2 对候选的原始 float 向量精排。候选规模取 $p = 100, 300, 500, 1000$ ，query batch 取 1, 128, 512, 2000。计时记录保留 H2D、GEMM、score D2H、候选选择与 CPU rerank 等分项，因此可以区分“GPU 计算快”与“端到端是否划算”这两个问题。

实验显示，异构方案的主要价值不是击败 GPU-only，而是暴露 CPU-GPU 分工的真实代价。query 数为 2000 时，GPU-only cublas_tree 的在线延迟为 78.68 us/query；异构 rerank 在 $p = 100$ 时达到 Recall@100=0.99999，但在线延迟为 924.10 us/query。分项时间显示，score D2H 拷贝为 389.43 ms，CPU 端候选选择为 1150.11 ms，CPU rerank 为 230.31 ms，均已超过 GEMM 本身的 49.72 ms。这里的候选选择是朴素 host 端实现：完整 2000×100000 score 矩阵先回传 CPU，再逐 query 选择 top- p 。若改为设备端 top- p 或至少用主机端 nth_element/partial sort 降低选择开销，绝对时间会下降，但仍无法改变“完整 score 回传过大”的架构结论。因此，若 GPU 已经能在设备端完成 Top- k ，把完整得分矩阵搬回 CPU 做精排并不划算；异构协同更适合候选集本来很小、或 GPU 端无法完成精排规则的场景。

方法	p	Recall@10	Recall@100	延迟 (us)	score D2H(ms)	CPU 后处理 (ms)
GPU-only	10	0.99995	—	78.68	17.38	0.00
GPU+CPU	100	0.99995	0.99999	924.10	389.43	1380.42
GPU+CPU	300	0.99995	0.99999	1109.27	319.27	1820.08
GPU+CPU	500	0.99995	0.99999	1769.27	501.12	2934.02
GPU+CPU	1000	0.99995	0.99999	2559.56	563.93	4416.60

表 15 GPU 粗排-CPU 精排在 query 数为 2000 时的分项结果

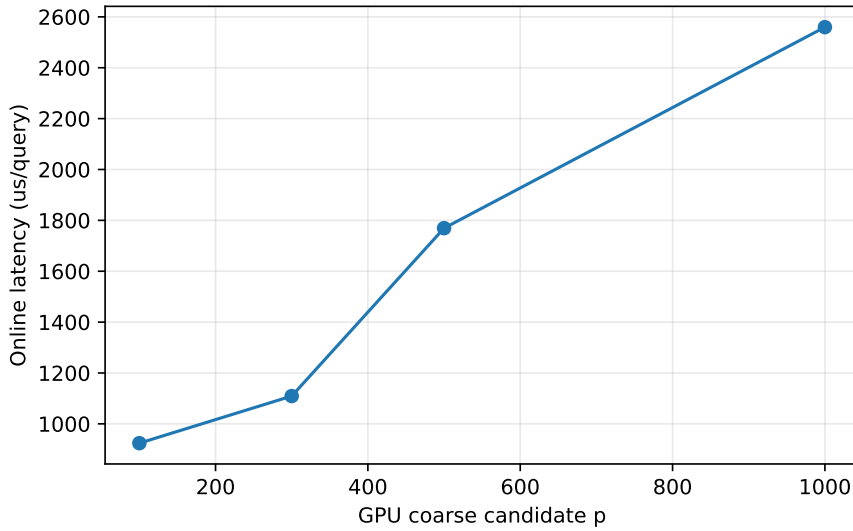


图 7 GPU 粗排-CPU AVX2 精排的候选规模与延迟关系

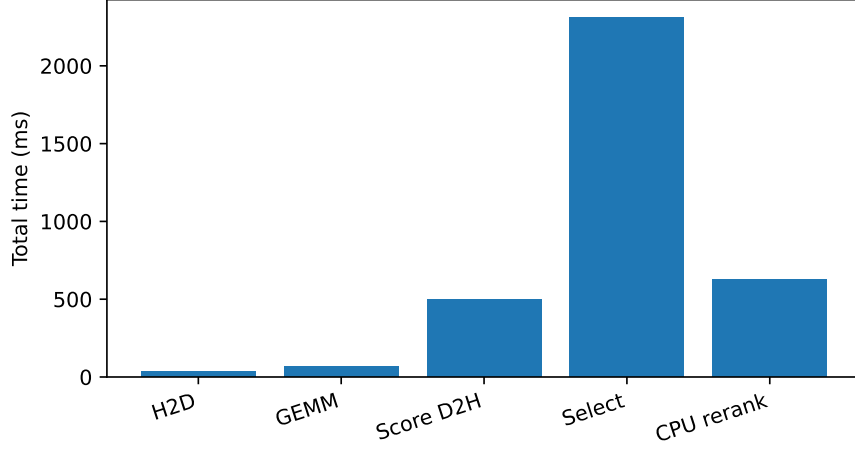
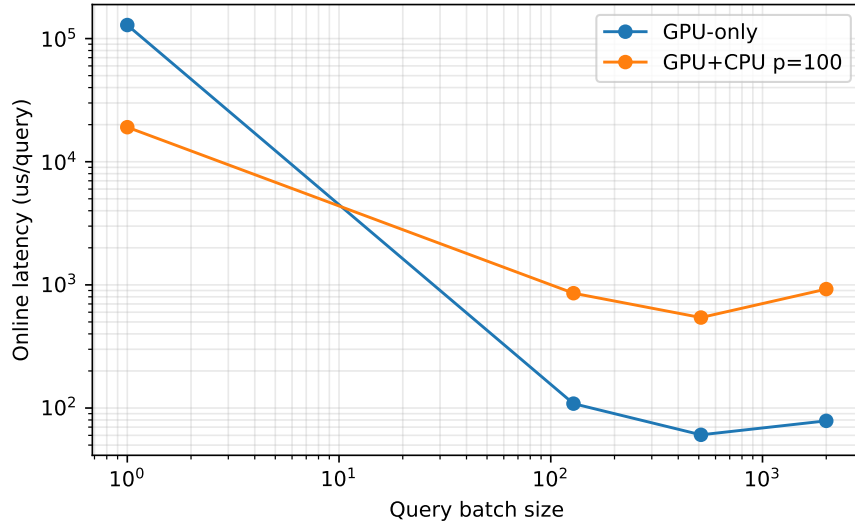
图 8 GPU+CPU 异构路径在 $p = 500$ 时的端到端时间分解

图 9 不同 query batch 下 GPU-only 与 GPU+CPU 异构路径的延迟变化

图 9 还显示 batch 尺寸对 GPU 路径影响很大。单 query 情况下，GPU-only 因 kernel 启动、数据搬运和固定开销导致单查询延迟很高；当 batch 增至 512 或 2000 后，base 矩阵可被批量共享，GEMM 吞吐优势才充分体现。异构路径虽然在 $p = 100$ 时保持很高召回，但它需要把 $query_n \times base_n$ 的完整 score 矩阵传回 CPU，再在主机侧做 top- p 选择。若未来要让异构协同真正有竞争力，关键不是把 CPU rerank 再向量化一次，而是在 GPU 端直接输出 top- p 候选 id，把 D2H 数据量从完整 score 矩阵降到 $query_n \times p$ 个候选。

异构实验给出了清晰的边界条件：跨 CPU-GPU 边界不应传递完整 score 矩阵，而应传递足够小的候选 id 和分数。若 GPU 端完成 top- p ，D2H 数据量可由 $Q \times N$ 个 float 降至 $Q \times p$ 个候选；当 $Q = 2000, N = 100000, p = 100$ 时，传输规模相差三个数量级。此时 CPU rerank 才可能作为灵活精排后端存在。当前实现保留完整 score 回传路径，用于量化朴素异构方案的代价，并给出设备端候选压缩的必要性。

GPU/异构方案	主要优点	主要风险
GPU Flat GEMM	规则矩阵乘, batch 大时吞吐高。	访问点最多, Top- k 和 score 存储压力大。
GPU grouped-IVF	减少访问点, 保留部分矩阵化。	分桶、小矩阵和 probe 不规则会降低利用率。
GPU-only cublas_tree	score 与 Top- k 均在设备端完成。	适合固定 Top- k , 复杂精排规则较难加入。
GPU coarse + CPU rerank	CPU 精排灵活, 可使用 AVX2 原始向量核。	若回传完整 score, D2H 和 host top- p 会主导延迟。

表 16 GPU 与异构方案的工程取舍

5 结果对比与瓶颈分析

5.1 四类后端的 Latency-Recall 对比

5.1.1 代表性结果

方法	平台/后端	Recall@10	延迟 (us)	加速比	说明
Flat-serial	i9 AVX2	1.00000	4571.62	1.00	串行 baseline。
Flat-AVX2	i9 AVX2	1.00000	1756.11	2.60	手写 AVX2/FMA 内积。
SQ-AVX2	i9 AVX2	0.99995	427.01	10.71	uint8 量化降低访存量。
IVF-PQ Pthread	i9 AVX2	0.95945	57.61	79.35	多核 IVF-PQ 代表点。
HNSW baseline	i9 AVX2	0.96865	65.48	69.82	单线程图索引代表点。
Budget-IVF-PQ	i9 AVX2	0.96520	186.54	24.51	query margin 调整 $nprobe$ 与 rerank p 。
MPI IVF-PQ 4x2	MS-MPI	0.96560	232.45	19.67	owner-computes + 定长候选协议。
MPI Block-HNSW 8x2	MS-MPI	1.00000	195.11	23.43	多 rank 图索引合并。
GPU cublas_tree	NVIDIA CUDA	0.99995	78.68	58.10	cuBLAS 得分 + 设备端树形 Top- k 。
GPU+CPU hybrid	RTX 4060 + i9	0.99995	924.10	4.95	$p = 100$, Recall@100 为 0.99999, 但 D2H 与 CPU 后处理较重。

表 17 代表性 ANN 实验结果汇总

算法/后端	Recall@10	AVX2 单线程	Pthread/OpenMP	MPI	CUDA GPU	口径说明
Flat	1.00000	1756.11	298.95	–	78.68	GPU 为 batch GEMM + 设备端 Top- k 。
IVF	~0.963	849.91	104.67	–	145.32	GPU 为 grouped-IVF, $nlist = 16, nprobe = 4$ 。
IVF-PQ	0.959–0.966	666.25	57.61	232.45	–	多核与 MPI 代表点召回不同, 仅比较量级。
HNSW/Block-HNSW	0.969/1.000	65.48	65.48 [†]	195.11	–	MPI 为 Block-HNSW 8 ranks $\times$ 2 threads。

表 18 不同后端上的代表延迟 (us/query, 同数据集、同串行 Flat baseline 归一; 各列代表点召回不完全相同, 不用于同召回精确排序)。[†]HNSW 多线程主要依靠 query 间并行提高吞吐, 单 query 图搜索路径依赖强, 表中保留同一单点延迟。

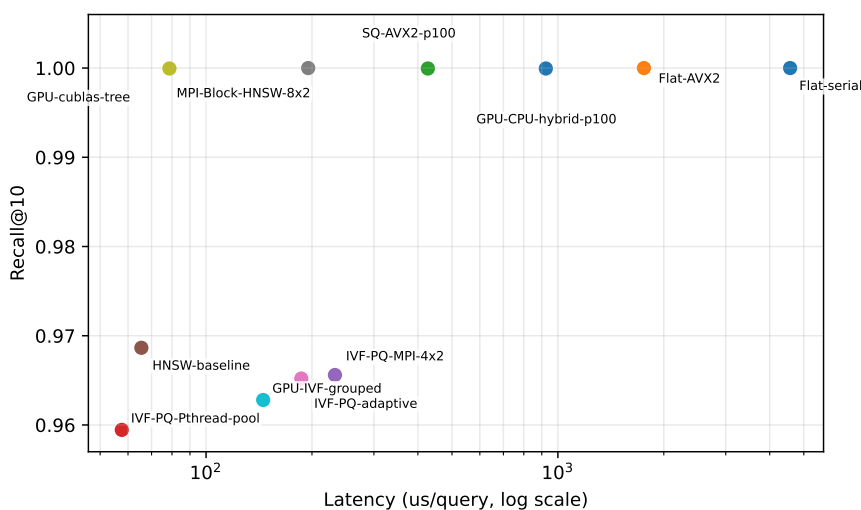


图 10 代表性算法的 Latency-Recall@10 横向对比

5.1.2 结果解读

表 17 显示,最快点并不必然是最适合的系统点。IVF-PQ Pthread 的延迟最低,但召回低于精确 Flat 与 GPU cublas_tree; 预算自适应 IVF-PQ 在同一 IVF-PQ 扫描空间中相对固定 4/1000 点同时降低延迟并提高召回,是 CPU 索引路径上的更优操作点; HNSW baseline 与 MPI Block-HNSW 补足了图索引在 trade-off 图中的位置,前者低延迟但 Recall@10 约 0.969,后者用多 rank 合并换取满召回; GPU cublas_tree 在高召回下保持较低延迟,但依赖 batch 与设备端 Top- k ; SQ-AVX2 在近似精确召回下显著降低延迟,是单机 CPU 上稳定的距离内核改进。OPQ/IVF-OPQ 将高召回区间继续向上推,但收益主要体现在同压缩率下的 recall,而非作为总加速比门面; GPU+CPU 异构 rerank 则证明跨设备协同必须把传输和候选选择纳入模型。trade-off 图的意义在于把这些点放到同一条 recall-latency 曲线上,而不是孤立比较加速比。

从工程推荐角度看,可以得到三个层次的选择。若目标是在 CPU 上获得稳定低延迟且允许一

定召回损失，多核 IVF-PQ 更合适；若固定在 IVF-PQ 路径内部继续调优，query 难度感知的预算分配比固定 $nprobe$ 更有效；若目标是高召回批量吞吐，GPU cublas_tree 是更直接的选择；若目标是研究压缩率固定时的召回上界，OPQ/IVF-OPQ 提供了比普通 PQ 更高的候选覆盖，但需要接受旋转和训练成本。异构 GPU+CPU rerank 的实验结果不作为最终最快方案，而作为一项约束证据：跨设备协同只有在候选压缩发生在设备端、回传数据量足够小时才可能收益明显。

表 18 的作用不是给出单列最优结果，而是把同一算法族在不同后端上的量级放在一起。Flat 在 AVX2 上体现距离内核优化，在 GPU 上体现矩阵化吞吐；IVF-PQ 在多核和 MPI 上体现候选削减与任务划分；HNSW 在单机和 MPI 上体现图索引的访问点削减；PQ/OPQ 则说明量化误差会影响同一候选预算下的召回。表中单线程基线用于标定各算法族的原始量级，各列代表点的 recall 和参数并不完全相同，因此它更适合看作量级对照，而不是同召回排行榜。不同点之间不是简单竞争关系，而是共同构成一张设计空间：系统可以选择更高召回、更低延迟、更低内存或更强吞吐，但很难同时极小化所有代价。

这些结果还给出一个反直觉结论：并行架构越强，越需要关注非计算项。AVX2 加速后，访存成为瓶颈；多核加速后，负载均衡成为瓶颈；MPI 分片后，rank-local 最慢路径成为瓶颈；GPU GEMM 加速后，Top- k 和 transfer 成为瓶颈。因此，实验分析不能停留在“把代码搬到更多硬件上”，还必须解释每次改变执行后端之后新的主导项在哪里。

5.2 Roofline 与瓶颈定位

5.2.1 Profile 证据链

Profile 采用计时、计数器与汇编互证的方法。CPU 侧以 VTune Top-Down 观察 Retiring、Memory Bound、Core Bound 与 LLC miss，并用 objdump 核对 AVX2/FMA 与查表路径；GPU 侧保留 CUDA event、cuobjdump/PTX-SASS，并在条件允许时补 Nsight occupancy、带宽利用与访存合并率。表 19 汇总了各内核的主要瓶颈。

Flat/IVF-PQ 热点循环包含 vmovups ymm、vfmadd231ps、vaddps、vhaddps 与 vextractf128，说明 x86 路径确实使用 256-bit AVX2/FMA 进行内积累加。GPU SASS 中可见 LDG.E.CI 与 FFMA 指令，说明 CUDA 路径的 score kernel 也落在全局加载加 FMA 的基本形态上。OPQ 实验额外保留 rotate、scan 与 rerank 分项，异构实验保留 H2D、GEMM、D2H、候选选择与 CPU rerank 分项，这使新实验可以直接放入同一瓶颈表，而不是只给总延迟。

内核	主瓶颈	证据与解释
Flat-AVX2	memory bandwidth	每个候选需读取 96 个 float，FMA 吞吐提升后访存成为主要限制。
SQ-AVX2	reduced memory traffic	uint8 表示降低向量读取字节数，收益来自访存压力下降。
PQ/ADC	LUT/cache pressure	距离计算变为 M 次查表累加，主瓶颈从连续 float 读取迁移到 code/LUT 访问。
IVF-PQ	list imbalance + rerank	候选数减少后，倒排表长度不均与精排候选维护更明显。
GPU GEMM	Top- k /transfer after GEMM	cuBLAS 加速得分矩阵后，设备端合并和数据搬运成为显性成本。
OPQ rotation	dense query transform	批量 query 旋转是额外稠密变换；单 query matvec 重用较低，因此只作为额外代价估算点。

表 19 Profile 证据中的主导瓶颈

Profile 证据的作用是防止把加速比解释成单一原因。例如，Flat-AVX2 相对串行 Flat 加速，不能简单归因于“用了 AVX2 所以快”；更准确的解释是，向量指令降低了每 96 维点积的指令数，但内存带宽占比上升，说明瓶颈已经从标量算术迁移到数据供应。SQ-AVX2 的更大收益也不能解释为整数运算更快，而应解释为压缩表示显著减少了读取字节数。类似地，GPU cuBLAS 加速 score

后，如果 Top- k kernel 和结果回传没有同步优化，整体延迟仍会停在后处理阶段。

OPQ 和异构实验的分项计时同样服务于这个证据链。OPQ 表中在线 latency 只统计扫描与精排，训练、构建和 base rotation 作为离线成本记录，这样才能比较 PQ 与 OPQ 的稳态查询代价。异构表中拆出 H2D、GEMM、score D2H、host top- p 和 CPU rerank，是为了说明慢点不在 GPU GEMM，而在完整 score 回传和主机端候选选择。没有这些分项，异构路径容易被误读为“CPU rerank 不够快”；有了分项后，瓶颈位置非常明确。

5.2.2 Roofline 定位

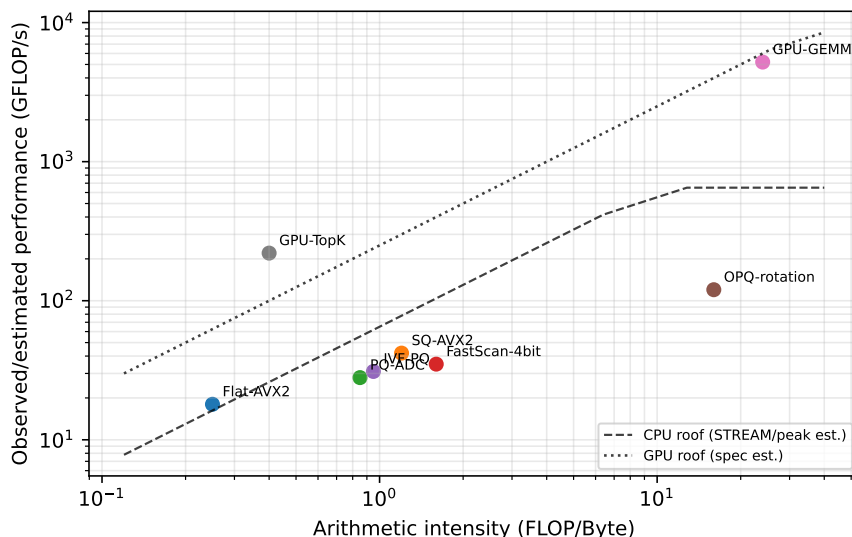


图 11 ANN 关键内核的 Roofline 定位（含 CPU/GPU 带宽斜线与峰值水平线）

Roofline[8] 的主要作用是解释“为什么下一步优化应改变数据表示或候选规模，而不是只堆更多指令”。Flat 距离计算的算术强度低，属于典型 memory-bound；SQ/PQ 通过压缩表示提高有效算术强度，但会引入查表和 cache 压力；GPU GEMM 算术强度高，接近 compute-throughput 区域，但 Top- k 合并与数据传输不具备同样规则性，因此成为后续瓶颈。

图 11 的屋顶线采用 CPU 侧 STREAM 量级带宽和估算峰值 FLOPS、GPU 侧设备规格量级带宽和估算峰值 FLOPS 绘制，仅用于相对定位。Flat-AVX2 的算术强度约为 0.25 FLOP/Byte，处于 CPU 带宽斜线附近；SQ 和 PQ 通过压缩表示把有效访存量降低，但查表和 code stream 使其仍偏向 cache/访存瓶颈；OPQ rotation 点按批量 query 旋转估算，显示该稠密变换的额外计算量上界；若只看单 query 的 96×96 matvec，矩阵重用不足，访存敏感性会更高。GPU GEMM 的算术强度最高，接近 GPU 计算屋顶区域；GPU Top- k 和 score 回传则重新落向低算术强度区域。该图与异构实验的分项计时相互印证：一旦把完整 score 矩阵从 GPU 搬回 CPU，系统重新落入低算术强度的数据搬运瓶颈。

Roofline 还可以解释为什么不同架构的优化顺序不同。CPU 侧 Flat 扫描处于低算术强度区域，优先级应是减少访存字节或减少访问点，因此 SQ、PQ、IVF 比继续打磨 FMA 更有价值。GPU 侧 full GEMM 的算术强度高，优先级则变为保持矩阵足够大、访存合并和设备端完成 Top- k ；如果过早把候选拆得很碎，反而可能丢失 GEMM 吞吐。MPI 侧不直接落在传统单机 Roofline 上，但其候选协议可以被理解为降低跨 rank 数据搬运强度：只传 top- k 候选而不是传原始向量或 score。

5.3 扩展性与代价模型

多核和 MPI 的扩展性可用 Amdahl 模型描述：

$$S(P) = \frac{1}{(1 - f) + f/P + \epsilon(P)},$$

其中 f 为可并行比例， $\epsilon(P)$ 表示调度、同步、cache 竞争或通信开销。Flat/SQ 距离扫描的 f 较高，但共享内存带宽会使 $\epsilon(P)$ 随线程数增加；IVF/HNSW 的 f 受候选不均和图路径依赖限制；MPI IVF-PQ 在单机多 rank 下 comm+merge 约 1%，因此 $\epsilon(P)$ 主要来自 rank-local 搜索时间差而非通信。GPU batch GEMM 更接近 Gustafson 式扩展：增大 batch 可以提高设备占用和 base 共享程度，但当 score 矩阵、Top- k 或 D2H 数据量随 batch 线性扩大时，吞吐收益会被后处理项抵消。

这一模型也解释了为什么“线程数、rank 数、batch 大小”不能被混为同一种并行参数。增加线程数共享同一内存层级，容易受到带宽和调度影响；增加 rank 改变数据所有权，需要候选协议保证正确性；增加 GPU batch 则扩大矩阵乘规模，提高硬件占用，但也线性增加 score 存储和 Top- k 工作量。三者都可能提高吞吐，却作用在不同代价项上。期末报告把它们放在统一表格中比较，是为了看清主导代价如何变化，而不是暗示这些并行度可以直接横向等价。

6 讨论与工程取舍

前文的横向评测给出了若干代表点，但更重要的是把这些点转化为系统设计原则。ANN 并行化通常不是一次性选择某个“最快算法”，而是在召回率、延迟、吞吐、内存、离线构建成本和硬件可用性之间做约束优化。SIMD 关注单点距离常数，多核关注共享内存调度，MPI 关注候选协议和全局合并，GPU 关注批量矩阵化和设备端后处理；四类约束共同决定系统的可用设计空间。以下进一步归纳参数旋钮、应用场景和工程边界。

6.1 参数旋钮的作用

ANN 系统中的参数通常具有两类作用：一类改变候选规模，另一类改变候选质量。IVF 的 $nprobe$ 、HNSW 的搜索宽度、PQ/OPQ 的 rerank 候选预算 p 直接改变候选规模；PQ 的子空间数 M 、OPQ 的旋转矩阵、FastScan 的 code 布局则更多改变候选质量或扫描常数。若把这些参数混在一起比较，很容易得到误导性结论。例如，增大 p 会提高 PQ 和 OPQ 的 recall，但这不能说明某个量化方式更好；只有固定 M 和 p 后，OPQ 相对 PQ 的召回提升才反映旋转前处理的作用。

参数旋钮之间还存在层级关系。距离内核参数位于最底层，例如 SIMD 展开、量化 scale、PQ 子空间数和 LUT 布局；索引参数位于中层，例如 $nlist$ 、 $nprobe$ 、HNSW entry 数和图扩展预算；并行参数位于外层，例如线程数、rank 数、batch size 和 GPU block 配置。底层参数改变单个候选的计算代价，中层参数改变候选集合，外层参数决定候选处理如何分摊到硬件。三类参数的耦合解释了为什么某些局部最优在系统层面并不最优。

以 IVF-PQ 为例，增大 $nprobe$ 会访问更多倒排表，提高召回；增大 p 会让 rerank 看到更多候选，也提高召回；增大线程数则试图降低这些候选的处理时间。若 $nprobe$ 很小，list 数量不足，intra-query 并行会缺少任务；若 p 很大，rerank 重新变成原始向量计算，PQ 粗排的收益被削弱；若线程数继续增加，共享内存带宽和局部 Top- k 合并会成为新瓶颈。因此，参数扫描不能只看一维曲线，而应把召回、候选规模和并行粒度一起解释。

GPU 路径的参数关系也类似。增大 batch size 可以提高 GEMM 利用率，但也增加 score 矩阵规模和 Top- k 工作量；增大 IVF 的 $nprobe$ 可以提高 recall，却会让 grouped-IVF 的分桶和小矩阵数量上升；增大异构 rerank 的 p 可以提高 CPU 精排候选覆盖，却会线性增加 D2H 候选或 score 回传压力。GPU 参数优化的核心不是单个 kernel 的最短时间，而是 score、Top- k 、transfer 三项之间的平衡。

参数类型	代表参数	主要收益	主要风险
距离内核	SIMD 展开、SQ scale、PQ M 、FastScan code layout	降低单候选距离估计成本。	访存、LUT/cache 压力或量化误差上升。
索引访问	IVF $nprobe$ 、HNSW 搜索宽度	提高候选覆盖与召回。	扫描候选增加，负载不均更明显。
精排预算	rerank p 、top- k 候选窗口	修正粗排误差，提高 Recall@K。	原始向量计算和候选维护开销上升。
并行粒度	线程数、rank 数、batch size	提高吞吐或降低平均延迟。	调度、同步、通信和数据搬运可能主导。

表 20 ANN 参数旋钮的作用

6.2 不同应用目标下的架构选择

同一组实验结果在不同应用目标下会产生不同结论。若目标是离线批量检索，GPU cublas_tree 的高召回和高吞吐更有吸引力，因为 batch 足够大时矩阵化收益可以覆盖固定开销。若目标是在线单 query 低延迟，GPU 的 kernel 启动和数据搬运不一定划算，多核 IVF-PQ 或 HNSW 更适合。若目标是内存受限的 CPU 检索，SQ、PQ、FastScan 和 OPQ 的压缩表示更重要；若目标是跨进程或跨节点扩展，MPI 候选协议的正确性和通信上界更重要。

这也说明“最优方法”必须带有场景限定。多核 IVF-PQ 在表 17 中延迟最低，但 Recall@10 不如精确 Flat 或 GPU cublas_tree；GPU cublas_tree 召回高且延迟低，但依赖足够 batch 和设备端 Top- k ；OPQ/IVF-OPQ 提高同码长召回，却不是最低延迟方案；MPI Block-HNSW 可以达到满召回，但其意义更多在于分布式候选合并能力。把这些点放入同一图中，不是为了选出唯一冠军，而是为了给不同约束下的选择提供依据。

对于 DEEP100K，底库规模相对较小，单机内存能够容纳完整数据，因此多核和 GPU 都有较好发挥空间；MPI 的通信占比也较低。若扩展到更大底库，结论会发生变化：GPU 全量 score 矩阵可能无法完整保留，IVF/HNSW 的访问点削减更重要；MPI 或多机分片会从功能验证变为实际容量需求；OPQ 的离线训练和 base 旋转成本也需要摊销到更长的索引生命周期中。本文没有把这些外推作为定量结论，但它们提供了理解现有结果的边界。

应用目标	更合适的路径	设计理由
高召回批量检索	GPU GEMM + 设备端 Top- k	batch 共享 base 矩阵，规则 score 计算吞吐高。
在线低延迟 CPU 服务	多核 IVF-PQ / HNSW	避免设备搬运，候选削减直接降低在线工作量。
内存受限检索	SQ/PQ/FastScan/OPQ	压缩表示降低访存和索引体积，OPQ 改善低码长召回。
分布式容量扩展	MPI owner-computes + 定长候选	数据所有权清晰，通信量与候选数而非底库规模绑定。
复杂业务精排	设备端 top- p + CPU rerank	仅在候选已压缩后，CPU 灵活精排才可能有意义。

表 21 不同应用目标下的架构选择

6.3 边界条件与慢点归因

综合实验中有两类结果同样重要：一种是成功加速，另一种是解释为什么某条路径不划算。Flat-AVX2 没有达到理论 8 倍加速，说明内存带宽限制真实存在；HNSW 多入口并行不一定更快，说明图搜索的数据依赖不能被简单线程化；MPI 通信占比低，说明定长候选协议有效，但也说明继续优化通信格式不是当前主要矛盾；GPU-CPU 异构慢于 GPU-only，说明完整 score 回传违背

了候选压缩原则。

这些结果限定了各条优化路径的适用边界。真实系统优化往往是在一个瓶颈被压低后暴露另一个瓶颈：SIMD 暴露访存，量化暴露 LUT/cache，IVF 暴露 list 不均，HNSW 暴露路径依赖，GPU 暴露 Top- k /transfer，MPI 暴露 rank-local 尾部。若只保留加速比最大的点，就无法解释这些迁移。异构协同的慢点说明，跨设备设计必须先满足候选压缩和低回传量这两个条件。

慢点归因也用于限定结论边界。OPQ 体现的是同码长召回增益，而不是所有配置都更快；异构实验量化的是 CPU-GPU 边界成本，而不是给出最终异构最优实现；Roofline 提供的是相对瓶颈定位，而不是绝对硬件峰值证明。这样处理后，结论更接近工程事实：少数关键现象被解释清楚，比单纯增加方法数量更有价值。

6.4 面向后续优化的路线

在现有结果基础上，后续优化应优先围绕“候选压缩发生在哪里”展开。CPU 路径上，OPQ 可以与 FastScan 进一步结合：OPQ 改善 code 的候选质量，FastScan 降低 code scan 的常数，两者位于 PQ 流水线的不同位置，理论上可以叠加。多核路径上，可以继续研究 list 长度感知的动态调度，使 IVF-PQ 在更大 $nprobe$ 下保持较低尾部等待。MPI 路径上，可以把 rank-local 搜索时间作为主要优化目标，而不是过早优化已经很小的通信比例。

GPU 路径的后续重点更明确：设备端 top- p 候选压缩是异构协同的前提。只有当 GPU 输出 $Q \times p$ 个候选 id 和分数，而不是完整 $Q \times N$ score 矩阵时，CPU rerank 才有可能发挥灵活精排价值。进一步地，GPU grouped-IVF 可以与设备端 top- p 结合，让每个 probe list 内先产生局部候选，再跨 list 合并。这样既保留访问点削减，也避免大规模 score 回传。

这些后续路线都沿着同一条 ANN 查询流水线展开。SIMD 提供距离内核和量化扫描基础，多核提供调度经验，MPI 提供候选协议原则，GPU 提供矩阵化和设备端 Top- k 基础；OPQ 与异构实验进一步把量化误差和设备边界纳入代价模型。下一步工作应继续沿着这条连接线深化。

6.5 局限

第一，OPQ 的额外旋转并非免费。实验中 OPQ 确实提高了 Recall@100，但它的主要价值是同码长召回增益，而不是替代并行后端成为最低延迟点；在线 latency 已按 scan+rerank 中位数报告，离线训练和 base 旋转不并入查询延迟。第二，GPU 粗排-CPU 精排并非天然优于 GPU-only；当 top- p 候选较大时，score D2H、CPU 候选选择与 rerank 会吞掉 GEMM 收益。第三，MPI 在 DEEP100K 规模下通信占比很低，因此继续优化通信协议的收益有限，rank-local 搜索与负载均衡更值得关注。

第四，固定 DEEP100K 与 $d = 96$ 使四架构之间具有可比性，但也限制了结论外推。更高维度数据会提高单点距离计算量，使 AVX2/GPU 的计算吞吐更重要；更大底库会放大 IVF/HNSW 的候选裁剪价值，也可能使 MPI 通信重新成为显性成本；更小 batch 则会削弱 GPU 的矩阵化优势。第五，本文的 OPQ 实现采用 PCA-balanced 旋转而非完整交替 Procrustes OPQ，因此结论应解释为“正交旋转量化前处理在当前实验框架中的收益”，而非对所有 OPQ 变体的上界断言。

本文仍有两个未展开方向。SDC 生产者-消费者流水线可尝试将编码下一批 query 与当前批查表累加重叠；HNSWPQ 可将 PQ 编码存入 HNSW 节点以压缩图索引体积。这两个方向均与当前主线相关，但若同时铺开会削弱 OPQ 与异构协同的深度，因此仅作为后续工作。

7 结论与实践认识

ANN 综合并行化的核心结论可以概括为瓶颈迁移。AVX2 将距离计算从标量算术迁移到内存带宽；SQ/PQ 将 full-vector 访存迁移到压缩 code 与 LUT；IVF/HNSW 将全库扫描迁移到候选选择与负载均衡；GPU 将得分计算迁移到 Top- k 和数据搬运；MPI 在当前规模下将通信问题弱化

为 rank 内搜索问题。OPQ/IVF-OPQ 与 GPU-CPU 异构实验正是围绕这条迁移链展开：前者试图用正交旋转降低量化误差，后者试图在 GPU 吞吐和 CPU 精排之间重新分配工作。这种系统视角更能解释不同架构的适用场景和优化边界。

定量结果显示：预算自适应 IVF-PQ 在同一实现内将固定 $(nprobe, p) = (4, 1000)$ 的 Recall@10 从 0.95945 提升到 0.96520，同时把 latency 从 229.06 us 降至 186.54 us，说明按 query 难度分配候选预算可以优于固定预算扫描。在 $M = 8, p = 1500$ 下，OPQ 将 PQ 的 Recall@100 从 0.95939 提升到 0.97316，在线 latency 与同码长 PQ 处于同一毫秒量级；IVF-OPQ 在 $nprobe = 8, p = 1500$ 下将 IVF-PQ 的 Recall@100 从 0.95228 提升到 0.96568。补充 $M = 12, 16$ 扫描显示，随着子空间数增加，普通 PQ 的召回本身接近饱和，OPQ 的边际收益随之下降。GPU-CPU 异构实验则给出一个明确的边界条件： $query_n = 2000, p = 100$ 时 Recall@100 达到 0.99999，但在线延迟约为 924 us/query，明显慢于 GPU-only cublas_tree 的 78.68 us/query，主要原因是完整 score 矩阵 D2H 和朴素 host 候选选择开销过大。这些实验共同说明，ANN 优化的关键不是把更多硬件简单串联起来，而是在候选规模、数据表示和设备边界之间选择正确的瓶颈迁移方向。

从并行程序设计角度看，ANNS 选题带来的核心训练是把“局部加速”提升为“端到端系统判断”。同一个距离函数在 AVX2、OpenMP、MPI 和 CUDA 上都可以并行化，但有效并行粒度并不相同：指令级并行要求内循环规则、数据连续并能被向量 lane 填满；线程级并行要求任务粒度足以覆盖调度与局部 Top- k 合并；rank 级并行要求候选协议能保持全局 id 与 recall 评估正确；设备级并行则要求计算被组织成足够大的规则矩阵，并尽量在设备端完成候选压缩。并行程序设计的难点不只是让更多执行单元工作，而是让数据布局、同步边界和算法依赖同时匹配硬件。

该选题还说明，性能优化必须建立在可解释的测量之上。只给 speedup 容易掩盖召回损失，只给 latency 容易掩盖离线构建和设备搬运，只给总时间容易掩盖瓶颈位置。因而，报告采用 Recall-Latency 曲线、profile 计数器、汇编/指令证据、Roofline 打点和分项计时共同约束结论。Flat-AVX2、SQ、PQ、IVF-PQ、HNSW、MPI 候选协议和 GPU GEMM 的结果并不是彼此独立的数字，而是同一条查询流水线在不同硬件边界下的观测值。最终形成的实践认识是：ANN 并行化的深度不来自方法数量，而来自能否解释每个方法压低了哪项代价、牺牲了什么指标、又把系统瓶颈推向何处。

参考文献

- [1] A. Babenko and V. Lempitsky. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. *CVPR*, 2016.
- [2] J. Sivic and A. Zisserman. Video Google: A Text Retrieval Approach to Object Matching in Videos. *ICCV*, 2003.
- [3] H. Jégou, M. Douze, and C. Schmid. Product Quantization for Nearest Neighbor Search. *IEEE TPAMI*, 2011.
- [4] T. Ge, K. He, Q. Ke, and J. Sun. Optimized Product Quantization. *IEEE TPAMI*, 2013.
- [5] Y. A. Malkov and D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE TPAMI*, 2018.
- [6] F. Andre, A.-M. Kermarrec, and N. Le Scouarnec. Cache Locality is Not Enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *PVLDB*, 2017.
- [7] J. Johnson, M. Douze, and H. Jégou. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 2021.
- [8] S. Williams, A. Waterman, and D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 2009.

[9] NVIDIA. cuBLAS Library User Guide, CUDA Toolkit Documentation.

A 数据口径与复核条件

正文图表采用统一数据集、距离函数和在线计时定义。所有进入横向比较的点均使用 DEEP100K, base 数为 100000, 维度为 96; 正式统计取前 2000 条 query, 召回率按同一 ground truth 文件计算。Latency-Recall 图中的代表点仅用于比较架构和算法族的量级, 不把不同 recall、不同候选预算的点解释为同召回排行榜。

预算自适应 IVF-PQ 与固定预算 IVF-PQ 的对比保持索引、PQ code、最终 rerank 距离函数和 query 集合一致, 只改变每类 query 的 $nprobe$ 与 p 。因此, 表 11 中的改进解释为候选预算分配变化, 而不是索引结构或距离函数变化。OPQ 与 PQ、IVF-OPQ 与 IVF-PQ 的对比必须保持三个条件一致。第一, 子空间数 M 、 $nprobe$ 和 rerank 候选预算 p 相同; 第二, 在线 latency 只统计 scan+rerank, 不把训练、base 旋转、query 旋转预处理或索引构建混入; 第三, 最终 rerank 均使用原始 float 向量计算内积距离。GPU-only 与 GPU-CPU 异构对比同时报告 online latency 和 H2D、GEMM、D2H、host top- p 、CPU rerank 等分项时间, 避免把计算、传输和主机端候选选择混成单一总数。

程序复核时遵循统一输入输出约定: 读取 DEEP100K 数据与 ground truth, 执行对应搜索函数, 并输出 average recall 与 average latency。附录仅限定数据、参数和计时口径, 保证正文分析与复核条件一致。

AI 使用报告

生成式 AI 主要用于技术问答、口径检查、 $\text{\LaTeX}$ 排版修正和文字一致性复核。实验数据、图表、延迟和召回率均来自程序输出、结果 CSV、profile 记录或重新生成的图表; AI 不用于生成或替代实验数据。下面按“提问节选 $\rightarrow$ 回复要点 $\rightarrow$ 判断与复核”的形式列出代表性 case。

A.1 OPQ 在线延迟口径

提问节选 > OPQ 相比 PQ 只多一次 96 维 query 旋转, 为什么表格中 OPQ/IVF-OPQ 可能比同配置 PQ 慢数倍? 是否把训练、base 旋转或索引构建计入了单查询延迟?

AI 回复要点 AI 建议把 OPQ 的成本拆成离线训练、base 旋转、query 旋转、PQ/IVF-PQ scan 和最终 rerank, 并用 warmup 后多次重复的 scan+rerank 中位数作为在线 latency。报告中不应把 OPQ 描述为更快的扫描器, 而应强调同码长、同候选预算下的召回增益。

判断与复核 该建议与 OPQ 的算法结构一致。结果文件已包含 train/build/rotate/scan/rerank 分项, 因此正文表 12 和表 13 只报告在线 scan+rerank 中位数, 离线成本单独列入表 14。最终表述改为“OPQ 与 PQ 在线延迟处于同一量级, 收益体现在同码长召回提升”, 避免用一次性成本污染单查询 latency。

A.2 预算自适应 IVF-PQ 是否构成更优操作点

提问节选 > 新加入的实验看起来没有超过前面几个最快点。能否从已有 IVF-PQ sweep 中找到一个真正更好的结果, 而不是只增加一个独立的小实验?

AI 回复要点 AI 建议不要把不同 runner、不同线程策略和不同 recall 口径的点直接相互宣称“更快”, 而应在同一 IVF-PQ 实现内部比较固定预算与按 query 难度分配预算的结果。如果自适应策略在同一索引、同一 query 集和同一 rerank 距离函数下同时提高 recall 并降低 latency, 就可以作为新的 Pareto 点进入主图。

判断与复核 结果文件显示 fixed 4/1000 的 Recall@10 为 0.95945、latency 为 229.06 us; adaptive conservative grouped 的 Recall@10 为 0.96520、latency 为 186.54 us, 平均 $nprobe = 3.91$ 、平均 $p = 919.20$ 。正文据此加入表 11 和图 4。同时保留边界说明: 多核 IVF-PQ 57.61 us 仍是横向汇总中的低延迟代表点, 自适应结果的结论限定在同一 IVF-PQ 参数空间内。

A.3 GPU 粗排-CPU 精排的慢点归因

提问节选 > GPU GEMM 很快, 为什么把 top- p 候选交给 CPU AVX2 rerank 后端到端反而变慢? 是否可以把这个结果写成架构结论?

AI 回复要点 AI 建议拆出 H2D、GEMM、score D2H、host top- p 选择和 CPU rerank, 而不是只报告总延迟。若完整 $Q \times N$ score 矩阵先回传 CPU, 再在主机端做候选选择, 则 D2H 和 host selection 会主导延迟; 更合理的方向是在 GPU 端直接输出 top- p 候选 id 和分数。

判断与复核 分项结果与该分析一致: $query_n = 2000, p = 100$ 时 GEMM 约 49.72 ms, 但 score D2H 为 389.43 ms, 朴素 host 候选选择为 1150.11 ms。正文因此没有把异构方案写成最终最快方法, 而是把它作为候选压缩位置的边界实验, 并在表 15 中保留分项时间。

A.4 代价模型公式的安全性

提问节选 > 报告中写 $T(q) = T_{\text{score}} + T_{\text{candidate}} + T_{\text{topk}} + T_{\text{sync}} + T_{\text{transfer}}$ 是否过于绝对? GPU stream、MPI 或流水线情况下这些项可能重叠。

AI 回复要点 AI 建议将公式改为近似分解, 并明确它不是严格运行时恒等式, 而是用于标记临界路径上的主要代价来源。单机 CPU 中 T_{transfer} 可近似为 0; GPU stream、MPI tile 或异步流水线中不同阶段可能重叠。

判断与复核 正文采用 $T_{\text{online}}(q) \approx \dots$ 的形式, 并在公式下方说明该式只用于瓶颈归因。这样既保留了分析框架, 也避免把阶段重叠问题写成错误的确定等式。

A.5 使用边界

AI 只作为技术资料索引器、一致性检查器和语言润色工具。所有数值结论均以可复核的结果文件和编译运行记录为准; 涉及口径变化的建议, 只有在代码逻辑、CSV 字段和图表重新生成结果相互一致时才进入正文。